



哈爾濱工業大學(深圳)
HARBIN INSTITUTE OF TECHNOLOGY, SHENZHEN

Asterinas EXT4 设计开发文档

面向 RustOS 的高性能强一致性 EXT4 文件系统

项目成员：

姓名	年级	联系方式 (QQ)
俞杰	大三	2113200249
梁丙煜	大三	1987247793
王毅航	大三	3223909812

指导教师：夏文、李诗逸

2026 年 06 月

目 录

摘要	1
1 项目介绍	3
1.1 项目背景及意义	3
1.2 项目目标	4
1.3 项目完成情况	5
1.4 国内外研究概况	5
2 需求分析与完成度对照	9
2.1 赛题要求理解	9
2.2 需求拆解	10
3 系统总体设计	11
3.1 设计目标与总体原则	11
3.2 总体架构	13
3.3 模块划分与职责	14
3.4 数据路径与恢复流程	15
3.5 关键机制设计	16
3.6 一致性边界与关键流程	17
4 关键模块设计与实现	20
4.1 EXT4 磁盘结构与 Extent 管理	20
4.2 JBD2 日志子系统	22
4.3 PageCache、mmap 与 O_DIRECT	24
4.4 并发控制	27
4.5 缓存体系	29
4.6 错误处理与健壮性设计	30
4.7 运行参数与可观测性	31
5 性能优化技术研究	32
5.1 实验口径	32
5.2 延迟归因方法	33
5.3 主要优化技术	34
5.4 量化结果	35
5.5 剩余差距分析	39
5.6 优化过程中的负结果	39
5.7 Rust 与 C 版 EXT4 的差异思考	40
6 系统测试与验证	42
6.1 测试环境	42

6.2	xfstests 功能正确性测试	42
6.3	xfstests 兼容性测试	44
6.4	崩溃一致性测试	44
6.5	并发正确性测试	45
6.6	性能测试	47
6.7	测试用例设计说明	47
6.8	测试结论分析	49
7	项目创新点	50
8	已知问题与后续工作	51
9	开发过程与工程管理	53
9.1	开发路线	53
9.2	代码组织与可维护性	54
9.3	遇到的典型问题	54
9.4	文档与代码版本管理	56
10	AI 使用说明	57
10.1	披露范围	57
10.2	AI 工具与模型清单	57
10.3	使用场景	57
10.4	人机分工	58
10.4.1	参赛队负责的工作	58
10.4.2	AI 辅助的工作	58
10.5	交互记录与可追溯性	58
10.6	诚信声明	58
11	总结	59
附录 A	核心实现细节补充	60
A.1	run_journaled_ext4 收敛点	60
A.2	DeviceBlockCache 的位置选择	60
A.3	WrittenCoverage 的不变量	61
A.4	fsync 链路的设计取舍	62
附录 B	复现命令与证据索引	62
B.1	常用复现命令	62
B.2	测试运行日志	63
参考资料		65

摘 要

本项目面向 2026 年全国大学生计算机系统能力大赛操作系统设计赛，在 Rust framekernel 操作系统 Asterinas 上设计并实现了一个 EXT4 文件系统，支持 POSIX 核心接口、Extent 连续块管理和 JBD2 日志机制，弥补了 Asterinas 在主流磁盘文件系统方面的能力空白。与仅以基础功能可用为目标的文件系统实现不同，本项目将一致性与性能之间的矛盾作为贯穿系统设计与后续优化的核心问题，围绕真实应用兼容性、崩溃一致性、并发一致性、缓存一致性与性能优化五个方面展开。

在真实应用兼容性方面，系统实现了文件与目录操作、Extent 映射、块分配等 POSIX 核心接口，并接入 Asterinas 的 PageCache、Vmo、bio 和块设备接口，打通 buffered I/O、mmap、fsync、O_DIRECT 以及并发读写等关键数据路径，使 fio、SQLite 等真实负载无需修改即可运行。在崩溃一致性方面，系统实现 JBD2 ordered 模式日志，将所有元数据修改收敛到统一事务入口提交，并在挂载时对已提交事务进行全量幂等重放，保证系统崩溃或掉电后磁盘可恢复到一致状态。

在并发一致性方面，系统将 per-inode 正确性锁升级为 `RwMutex`，明确区分会改变文件结构的操作与纯数据覆盖写，在多文件及同文件并发读写下保持数据与元数据一致。在缓存一致性方面，系统以写回与失效协议统一 buffered I/O、mmap 与 O_DIRECT 的数据视图，并以清晰的状态边界隔离 DeviceBlockCache 与 JBD2 overlay，避免页缓存与磁盘内容分叉。在性能优化方面，项目在不破坏上述一致性语义的前提下，结合四层延迟归因与 EXT4/EXT2/ramfs 三种文件系统横向对照定位瓶颈，实现 DeviceBlockCache、WrittenCoverage、Extent 映射缓存、连续脏页批量写回以及 O_DIRECT 共享锁覆盖写等优化，减少重复的元数据读取、映射查询和不必要的锁串行化开销。

本项目主要围绕以下四个目标展开：

- **目标 1：完成 Asterinas EXT4 主体功能。**

实现 POSIX 核心接口、目录操作、Extent 管理、inode 与块分配，以及 VFS 和块设备适配。

- **目标 2：实现 JBD2 日志与崩溃恢复。**

支持 transaction、handle、commit、checkpoint 和 recovery，并维护 ordered 模式下的数据写回与日志提交顺序。

- **目标 3：与 Linux EXT4 的性能对比。**

使用 FIO、filebench 或其他测试工具评估文件系统基本性能，使用 SQLite 或其他应用测试文件系统在真实工作环境下的性能表现。

- 目标 4：完成性能创新优化。

围绕 Rust 语言特性、Asterinas 平台边界以及实际测试中暴露的性能瓶颈开展优化。

目前，JBD2 崩溃恢复测试完成 9 个场景两轮测试 18/18 PASS，host-crash fsync 持久化测试达到 4/4 PASS；并发正确性测试 7/7 通过，xfstests concurrency 测试 10/10 通过。在性能方面，`fio O_DIRECT` 顺序读写在 4 KiB 至 1 MiB 块大小下达到 Linux EXT4 的 90% 以上；同文件并发写在 `numjobs=2/4` 时分别达到 Linux EXT4 的 165% 和 187%。

对于 SQLite speedtest1 真实应用负载测试，经过优化，墙钟时间由约 2010.7 s 降低至 234.9 s，整体性能提升约 8.6 倍，

项目主要目标的完成情况概括如下。

项目主要目标完成情况

编号	目标	完成情况	说明
1	EXT4 主体功能	基本完成	POSIX 核心接口、Extent、目录操作和 VFS 集成已实现。
2	JBD2 与崩溃恢复	基本完成	已完成多场景验证测试。
3	与 Linux EXT4 的性能对比	基本完成	<code>fio</code> 测试的 I/O 性能达到 Linux EXT4 的 90% 以上，并发同文件写在 <code>nj=2/4</code> 时达到 Linux 的 165%/187%。
4	性能创新优化	初步完成	SQLite 相比初始版本提升约 8.6 倍， <code>fio</code> 的部分 I/O 性能超越 Linux。

综上，本项目已经完成 Asterinas EXT4 的主体功能、日志恢复、缓存一致性和并发访问路径，并在保持数据正确性和崩溃恢复语义的前提下，显著改善了合成负载、并发负载和真实数据库负载的性能。

1 项目介绍

1.1 项目背景及意义

在云原生、企业级数据库、大数据与 AI 训练等场景中，文件系统作为数据持久化与访问的核心载体，其行为正确性与性能直接决定着上层服务的可用性与运维成本。以 MySQL、PostgreSQL 等数据库为例，它们依赖底层文件系统稳定运行来实现事务的 ACID 语义，一旦在崩溃或掉电后出现元数据不一致，就可能导致事务回滚、数据损坏乃至业务中断；而对象存储、消息队列等服务则对文件系统在高并发、大吞吐下的稳定性与延迟提出了更苛刻的要求。因此，一个真正可用于生产场景的文件系统，必须同时在真实应用兼容性、崩溃一致性、并发一致性、缓存一致性与 I/O 性能五个方面达到要求。

然而，这五个目标却难以同时满足。在崩溃一致性方面，Linux EXT4 采用 JBD2 日志来保证崩溃恢复能力，但日志机制会带来碎片化的元数据 I/O、严格的写入顺序约束和频繁的设备 flush，在 I/O 关键路径上引入诸多串行节点；随着 SSD、NVMe 等高速设备的普及，存储介质本身的延迟大幅下降，这些由崩溃恢复机制引入的软件开销反而日益成为新的性能瓶颈。在缓存一致性方面，为发挥页缓存的加速能力，buffered I/O、mmap 与 `O_DIRECT` 必须共享一致的数据视图，否则极易在掉电或并发场景下读到陈旧数据。在并发一致性方面，多线程对同一文件或目录的读写会同时争用页缓存、inode 元数据与日志事务，稍有不慎便会破坏数据完整性或引入难以复现的竞争。如何在不牺牲崩溃一致性、并发一致性与缓存一致性的前提下，尽可能逼近高速设备的真实性能上限，已成为现代存储系统设计中的核心难题。

另一方面，传统操作系统与文件系统大多以 C 语言实现，手动内存管理容易引入越界、悬垂指针、重复释放等问题，给文件系统这类长期运行、状态复杂、并发度高的子系统带来额外的可靠性风险。Asterinas 是一个基于 Rust 的 framekernel 操作系统，它通过 OSTD 将可信计算基缩小，把内核服务运行在受约束的非特权环境中，同时兼容 Linux ABI。这一平台为文件系统设计带来两重契机：其一，借助 Rust 的类型系统、所有权模型与编译期借用检查，从根本上降低内存安全与数据竞争风险，使复杂的并发与缓存逻辑更易于推理；其二，在 framekernel 重新划定的架构边界下，文件系统运行于非特权环境、无法随意使用特权指令与传统内核内存接口，这既带来约束，也促使我们重新审视存储层的一致性保障与性能优化空间。

但 Asterinas 目前缺少主流磁盘文件系统支持，仅能依赖 ramfs 等内存文件系统，

这严重限制了它在真实应用、数据库与云原生场景下的可用性，也使其难以与 Linux 进行同口径的存储性能对比。在所有候选中，EXT4 是 Linux 生态中部署最广、生态最成熟的磁盘文件系统之一，既能直接承载 fio、SQLite 等大量真实负载，又具备 Extent、JBD2 日志等值得深入研究的现代特性，并可与 Linux EXT4 形成天然的对照基线。正是为了在上述五个目标的共同约束下系统性地探索 RustOS 存储层的设计空间，本项目才选择在 Asterinas 上实现一个强一致、高性能的 Rust EXT4。这不仅是完成比赛题目，也为 RustOS 的存储层建设提供了一个覆盖兼容性、一致性与性能的、相对完整的工程案例与方法参考。

1.2 项目目标

基于上述背景，本项目并非以“移植一个可用的 EXT4”为终点，而是围绕真实应用兼容性、崩溃一致性、并发一致性、缓存一致性与 I/O 性能优化五个目标，探索在 RustOS 受约束运行环境中协调这些目标的工程方法。为此，本项目选择在 Asterinas 操作系统上实现一个兼容 POSIX、支持 EXT4 核心磁盘格式与 JBD2 崩溃一致性语义的高性能文件系统，并在标准化实验环境中对其正确性、可靠性和性能进行系统验证。围绕上述五个目标，具体目标如下：

- **真实应用兼容性**：实现 create、open、close、read、write、lseek、stat、truncate、mkdir、rmdir、unlink、rename 等 POSIX 核心接口，并完成与 VFS、块设备层的适配，使 fio、SQLite 等真实负载无需修改即可直接运行。
- **崩溃一致性**：实现 EXT4 Extent 连续块管理与 JBD2 ordered 模式日志，支持事务管理、日志刷盘、checkpoint 与挂载时全量崩溃恢复，保证在系统崩溃或掉电后磁盘可恢复到一致状态。
- **并发一致性**：支持多文件及同文件并发读写，在并发数据路径上保持数据与元数据的一致性，并通过确定性校验验证其正确性。
- **缓存一致性**：将 buffered I/O 与 mmap 接入 PageCache/Vmo，由 fsync 统一完成脏页写回、事务提交与设备 flush，并保证 `O_DIRECT` 与页缓存之间的数据视图一致。
- **I/O 性能优化**：在 QEMU/KVM + virtio-blk 标准环境下与 Linux EXT4 进行同口径性能对照，定位并消除一致性机制在 I/O 关键路径上引入的额外开销，尽可能逼近高速设备的性能上限；并形成面向 RustOS/framekernel 的可复用优化方法，给出量化数据与研究结论。

1.3 项目完成情况

目前，本项目已经完成 POSIX 核心接口、EXT4 Extent 管理、JBD2 日志、崩溃恢复、PageCache、mmap、O_DIRECT 和并发读写等主体功能，并通过 xfstests、fio 和 SQLite 等负载进行了验证。各方向的具体完成情况如表 1-1 所示。

表 1-1 项目完成情况

方向	完成情况	说明
功能完整度	优秀档主体功能已大部分完成，已通过大量 xfstests 测试验证功能（85%）。	POSIX 核心接口、Extent、JBD2、fsync/fdatasync、PageCache、O_DIRECT、mmap 和并发读写均已实现并测试。
崩溃恢复	已完成多场景验证（90%）。	JBD2 crash matrix 18/18 PASS，host-crash fsync 4/4 PASS。
性能	合成负载接近或部分反超 Linux（85%）。	fio O_DIRECT 读写达到 Linux EXT4 的 90% 以上；并发同文件写在 nj=2/4 时达到 Linux 的 165%/187%，SQLite 的性能相较于开始提升 8.6 倍。
创新性优化	探索多种结合 Asterinas 平台特点和 Rust 实现方式的优化方法（50%）。	围绕 Rust 语言特性、Asterinas 平台边界以及实际测试中暴露的性能瓶颈开展优化。

1.4 国内外研究概况

EXT4 是 Linux 生态中长期使用的主流本地文件系统。它并不是从零设计的新文件系统，而是在 EXT3 已有磁盘格式、日志机制和工具链基础上逐步演进形成的。早期 EXT4 研究工作的主要目标，是在保持较好兼容性和可靠性的同时，解决 EXT3 在大容量磁盘、连续空间管理和大文件访问方面的扩展性问题。为此，EXT4 引入了 Extent 映射、48 位块号、更大的文件系统容量、持久化预分配和更灵活的块组组织方式 [1]。

从工程实现角度看，Linux EXT4 的性能并不是由某一个局部函数决定的，而是由多种机制共同构成。Extent 使用一个映射项描述连续逻辑块到连续物理块的关系，降

低大文件和顺序访问中的映射元数据开销；multi-block allocator (mballoc) 倾向于一次分配较大的连续区域；delayed allocation (delalloc) 则将物理块分配推迟到脏页写回阶段，使文件系统能够综合多个页面的写入需求后再选择物理位置 [3, 2]。这些机制与 PageCache、writeback、预读、块设备调度和日志提交相互配合，共同决定普通 buffered I/O 和数据库负载中的实际性能。

Linux VFS 为不同文件系统提供了统一的系统调用与内核对象接口。open、read、write、stat 和 mmap 等用户接口首先进入 VFS，随后再通过 inode、file、dentry 和 superblock 等对象分派到具体文件系统实现 [4]。因此，一个文件系统即使能够正确解析 EXT4 磁盘格式，也不能直接等同于完整的内核文件系统。它还必须适配目标操作系统中的路径解析、文件对象生命周期、页缓存、内存映射、并发锁和块设备接口。这也是用户态 EXT4 库与内核态 EXT4 实现之间的重要区别。

在崩溃一致性方面，Linux EXT4 主要依赖 JBD2 组织元数据事务。其默认的 ordered 模式只将文件系统元数据写入日志，但要求相关普通数据块在对应元数据提交之前先写入 home block。系统恢复时根据完整的 commit 记录判断事务是否已经提交，并对已经提交但尚未完成 checkpoint 的元数据进行重放 [2, 5]。这种设计避免了将所有普通文件数据写入日志所产生的高额写放大，同时又比完全不约束数据顺序的 writeback 模式提供更强的恢复语义。

然而，日志在提供崩溃恢复能力的同时，也是以一定的运行性能为代价换取一致性保证，其本身正是一致性与性能之间矛盾的集中体现，而非这一矛盾的消解。事务提交需要 descriptor、metadata payload、commit block 以及设备 flush 按照规定顺序到达稳定存储；频繁 fsync 会放大日志提交和设备屏障成本；延迟分配虽然能够减少小粒度元数据更新，但又要求系统正确维护脏页、空间预留、写回失败和 ENOSPC 语义。写时复制文件系统可以避免原地覆盖，但通常需要承担额外的空间占用、树结构更新和垃圾回收成本。可见，现代文件系统的各类一致性机制并未真正消除上述矛盾，而只是在持久化语义、写放大、缓存复杂度和运行性能之间，选择了不同的权衡方式。

已有研究也表明，成熟文件系统仍需要持续处理大量语义错误、并发错误和异常路径问题。针对 Linux 文件系统长期演进过程的研究发现，文件系统补丁中存在大量维护、错误修复、性能和可靠性改进，其中语义错误、并发同步和失败处理路径尤其难以完全消除 [6]。这说明实现一个可运行的文件系统只是第一步，长期可靠性还依赖系统化测试、故障注入、边界检查和回归验证。

随着 Rust 在系统软件中的应用不断增加，研究者开始探索如何利用所有权、生命周期和类型系统改善操作系统及文件系统的安全性和可维护性。Theseus 尝试使用 Rust 的语言级机制表达操作系统组件边界和状态关系，将部分系统不变量交由编译器检查

[9]。Bento 则提出了面向 Linux 内核文件系统的安全 Rust 开发框架，希望在保持较低运行开销的同时改善文件系统开发、调试和升级效率 [7]。

在崩溃一致性方向，SquirrelFS 进一步使用 Rust typestate 表达持久化内存文件系统中的元数据更新顺序，将部分崩溃一致性约束转化为编译期类型检查 [8]。这些研究说明，Rust 的价值不仅体现在减少悬空指针、越界访问和重复释放等内存安全问题，也可以用于表达对象状态、操作阶段和更新顺序。但是，类型系统能够保证哪些性质，仍然取决于实现者是否将相应的不变量正确编码到接口和类型中。

因此，Rust 并不会自动提高文件系统性能，也不会自动解决崩溃一致性问题。JBD2 的事务边界、commit block 前的持久化顺序、revoke 语义、fsync 与 PageCache 的协同，以及 O_DIRECT 与 buffered I/O 的缓存可见性，本质上仍然属于文件系统协议和系统架构问题。Rust 提供的是更加清晰的所有权边界、错误传播方式和状态表达能力，真正的性能与一致性仍需要针对 I/O 路径、缓存、锁和日志机制进行设计。

Asterinas 为本项目提供了与 Linux 内核不同的运行环境。Asterinas 采用 framekernel 架构，并通过 OSTD 提供面向安全 Rust 内核开发的基础设施，同时保持对 Linux ABI 的兼容 [10, 11]。对于 EXT4 而言，这意味着系统不能直接复用 Linux 中成熟的 address_space、writeback、JBD2 和 block layer 实现，而需要重新适配 Asterinas 的 VFS、PageCache、Vmo、bio、同步原语和 virtio-blk 设备路径。

开源 `ext4_rs` 为本项目提供了 EXT4 磁盘格式解析、inode、目录项和基础 Extent 操作的实现基础 [12]。但该项目主要定位为与具体操作系统相对独立的 Rust EXT4 库，并不完整包含本项目所需的 JBD2 事务运行时、内核 VFS 集成、PageCache 协同、并发控制和崩溃恢复机制。因此，本项目并不是简单地将已有库接入 Asterinas，而是在其磁盘格式基础上补齐一个内核文件系统所需的运行时语义。

表 1-2 对主要研究与工程对象进行了总结。

表 1-2 相关研究与工程对象对比

研究或工程对象	主要特点	本项目的借鉴或区别
Linux EXT4	成熟的 Linux 本地文件系统，具有 Extent、JBD2、delalloc、mballoc、PageCache 和 writeback 等完整机制。	作为文件系统语义、崩溃恢复和性能测试的主要对照对象，但其内部实现与 Linux 内核基础设施深度耦合，无法直接移植到 Asterinas。

续下页

表 1-2 相关研究与工程对象对比（续）

研究或工程对象	主要特点	本项目的借鉴或区别
Bento	面向 Linux 内核的安全 Rust 文件系统开发框架，关注开发效率、故障隔离、用户态调试和在线升级。	说明 Rust 可以用于构建接近原生内核性能的文件系统，但其目标不是实现兼容现有 EXT4 磁盘格式的完整文件系统。
SquirrelFS	使用 Rust typestate 检查持久化内存文件系统中的元数据更新顺序和崩溃一致性约束。	其类型化持久化顺序具有启发意义，但面向持久化内存和软更新机制，与本项目的块设备 JBD2 日志路径不同。
Theseus	使用 Rust 和语言内设计方法表达操作系统组件边界、状态依赖和部分系统不变量。	为使用类型和所有权组织内核运行时状态提供参考，但并不以 Linux ABI 和 EXT4 兼容为主要目标。
开源 ext4_rs	使用 Rust 编写的操作系统无关 EXT4 实现，提供磁盘结构和部分文件、目录及 Extent 操作。	复用其磁盘结构基础，并新增 JBD2、内核 VFS 集成、PageCache、并发控制、缓存管理和崩溃恢复机制。
Asterinas	采用 Rust framekernel 和 OSTD 的操作系统，强调小型且可靠的内存安全 TCB，并兼容 Linux ABI。	作为本项目的运行平台，要求 EXT4 重新适配其 VFS、Vmo、PageCache、bio、virtio-blk 和非特权运行边界。

综合来看，本项目的定位不是重新设计一种全新的磁盘文件系统，而是在 RustOS 和 framekernel 环境中实现主流 EXT4 文件系统的核心能力，并研究成熟 Linux 文件系统机制在新平台中的适配方式。本项目一方面继承 EXT4 的磁盘格式、Extent 和 JBD2 语义，另一方面需要重新组织 Asterinas 环境下的缓存、锁、事务和块设备路径。

这一定位也与赛题的“学术型”要求相吻合：除交付可运行的系统代码外，还需要说明系统为什么采用当前架构，哪些 Linux 机制可以复用其设计思想但不能直接复制实现，探索 1-2 种面向 RustOS/星绽架构的 EXT4 性能优化技术，以及测试数据如何支持相应的设计和性能判断。

2 需求分析与完成度对照

2.1 赛题要求理解

赛题要求在星绽 OS / Asterinas 的 framekernel 架构上设计适配 RustOS 的 EXT4 文件系统，并按照 xfstests 规范验证正确性。同时，赛题不是只要求“能挂载、能读写”，还要求探索 RustOS 下的性能优化技术，使用 fio、SQLite 等工具评估性能，最后形成研究结论。

从评分权重来看，项目需要同时对齐实现完整度、文档完整性、Demo 质量和创新性四个方面。其中，实现完整度和 Demo 质量决定项目是否具备稳定运行和展示的基础；文档完整性和创新性则决定项目的设计思路、工程工作量和研究价值能否得到充分体现。赛题评分要求与本项目材料之间的对应关系如表 2-1 所示。

表 2-1 赛题评分要求与本项目材料对应关系

评分项	权重	优秀档要求	本项目对应材料
实现完整度	40%	完成 JBD2、事务管理、日志刷盘、全量崩溃恢复和多文件并发访问，保证并发过程无数据错乱，xfstests 核心测试通过率不低于 95%。	第 3-4 章介绍系统架构与关键模块设计，第 6 章给出功能测试、崩溃恢复测试和并发正确性验证结果。
文档完整性	20%	提供架构设计、用户手册、测试文档、差异化分析以及 Rust/C EXT4 性能优化研究报告。	本文对设计、实现、测试、优化和差异分析进行统一汇总，同时保留各类独立文档作为补充材料。
Demo 质量	20%	系统运行稳定，性能达到 Linux EXT4 的 90% 及以上，优化技术带来的提升不低于 5%，并发访问过程中无数据问题。	第 5-6 章给出 fio、SQLite、并发测试数据；对暂未达到性能要求的测试项目进行客观归因。

续下页

表 2-1 赛题评分要求与本项目材料对应关系（续）

评分项	权重	优秀档要求	本项目对应材料
创新性	20%	探索 1-2 种面向 RustOS 的文件系统优化技术，保证测试数据可信，并形成可复用的研究方法。	第 5 章和第 7 章总结缓存、预分配、共享锁以及 profiling 等优化技术和性能分析方法。

2.2 需求拆解

根据赛题目标和评分要求，本项目将系统需求拆分为四个层次。第一层是文件系统基础功能，要求应用可以按 POSIX 方式创建、删除、读写文件和目录。第二层是 EXT4 磁盘格式，要求文件映射、inode、目录项、位图和块组等结构能够正确读写。第三层是强一致性，要求系统崩溃后可以通过日志恢复到自洽状态。第四层是性能与研究，要求用定量实验解释差距并证明优化有效。上述四层需求之间具有逐层递进关系，如图 2-1 所示。

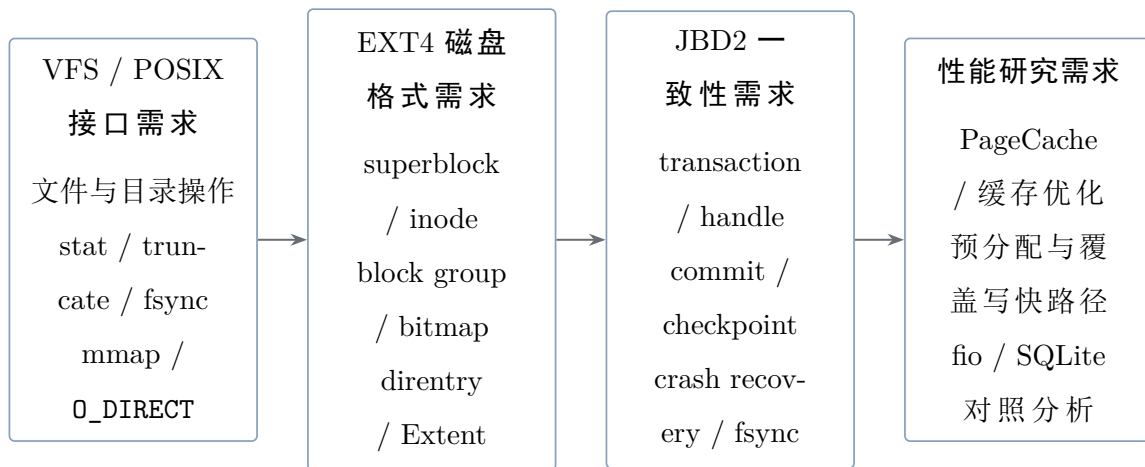


图 2-1 赛题需求分层图

在 VFS 与 POSIX 接口层，项目主要完成文件和目录相关的系统调用语义，包括普通文件读写、目录创建与删除、路径查找、stat、truncate、fsync、mmap 以及 O_DIRECT 等能力。这一层主要位于 `kernel/src/fs/ext4/inode.rs` 和 `kernel/src/fs/ext4/fs.rs` 中，负责将 Asterinas 内核中的 VFS 请求转换为 EXT4 文件系统内部操作，并处理 inode 生命周期、页缓存协同、直接 I/O 路径以及同步语义。

在 EXT4 核心结构层，项目围绕 EXT4 磁盘格式实现了 superblock、block group、inode、directory entry、Extent 树、块分配和 inode 分配等基础功能。相关代码主要位于 `kernel/libs/ext4_rs/src/ext4_defs` 和 `kernel/libs/ext4_rs/src/ext4_impls`。

其中, `ext4_defs` 负责定义与磁盘格式对应的数据结构, `ext4_impls` 负责实现目录操作、Extent 查找与更新、空间分配、文件读写和元数据更新等逻辑。通过该层, 系统能够按照真实的 EXT4 磁盘格式组织文件数据和元数据, 而不是使用简化的自定义文件系统格式。

在 JBD2 日志一致性层, 项目新增了 `handle`、`transaction`、`commit`、`checkpoint`、`recovery` 和 `journal space` 管理等模块, 用于保证 EXT4 元数据更新过程中的崩溃一致性。相关代码主要位于 `kernel/libs/ext4_rs/src/ext4_impls/jbd2`。该层采用 `ordered` 模式, 数据块直接写入 `home` 位置, 元数据先进入 `journal`, 提交时按照 `descriptor`、`metadata payload`、`flush` 屏障和 `commit block` 的顺序落盘。系统重启后, `recovery` 逻辑只重放已经完整提交的事务, 从而保证元数据更新不会处于半完成状态。

在性能优化层, 项目围绕 SQLite、`fio` 和并发测试中暴露的瓶颈开展了多轮优化, 主要包括设备块缓存、PageCache 协同、批量 `checkpoint`、Extent 映射缓存、预分配支持、`O_DIRECT` 覆盖写共享锁快速路径以及 `profiling` 延迟归因机制。相关实现分布在 `kernel/src/fs/ext4/fs.rs` 以及 `ext4_rs` 中的 `file`、`extents` 和 `jbd2` 等模块。通过这些优化, 系统在维持崩溃一致性语义的同时, 减少了重复 Extent 查找、频繁刷盘以及同文件并发写串行化带来的额外开销。

3 系统总体设计

3.1 设计目标与总体原则

本项目面向 Asterinas 操作系统设计并实现一个支持 EXT4 磁盘格式和 JBD2 崩溃一致性语义的文件系统。系统不仅需要完成文件创建、删除、读写、目录管理和空间分配等基本功能, 还需要适配 Asterinas 现有的 VFS、PageCache、Vmo、`bio` 和块设备接口, 在保证文件系统正确性与可恢复性的基础上, 支持 `buffered I/O`、`mmap`、`O_DIRECT`、`fsync` 等实际应用所依赖的访问方式。

与单纯解析 EXT4 磁盘镜像不同, 本项目需要同时处理磁盘格式、缓存一致性、事务提交、并发控制和块设备持久化顺序等问题。EXT4 的目录项、`inode`、位图、`extent` 和块组计数之间存在复杂关联, 一次文件系统操作通常会修改多个磁盘结构。如果这些修改缺少统一的事务边界, 系统在异常断电或内核崩溃后可能出现目录项存在但 `inode` 未分配、数据块已经分配但 `extent` 尚未更新等不一致状态。因此, 本项目将崩溃一致性作为总体设计的首要约束, 并通过 JBD2 日志机制组织元数据更新。

系统总体设计遵循以下原则。

第一, 采用分层解耦的架构, 将平台相关的内核集成逻辑与 EXT4 磁盘格式逻辑

分离。Asterinas 的 VFS 对象、页缓存、内存映射、bio、锁和启动参数由集成层负责；superblock、inode、目录项、extent、位图以及 JBD2 事务等核心语义由文件系统核心库负责。通过这种划分，EXT4 核心逻辑可以相对独立地进行验证，平台相关的一致性和并发问题也能够集中在明确的边界内处理。

第二，所有元数据修改必须具有统一的事务入口。文件创建、删除、重命名、截断、空间分配等操作虽然会修改不同类型的磁盘结构，但均应收敛到统一的 journaled operation 入口。该入口负责创建或加入 JBD2 事务、记录被修改的元数据块、处理提交错误，并在操作完成后执行必要的缓存失效。统一事务入口可以避免日志逻辑分散在不同 VFS 路径中，也为性能统计、故障注入和一致性测试提供稳定边界。

第三，采用明确的缓存一致性协议。系统同时存在 PageCache、Vmo、设备块缓存、extent 映射缓存和 JBD2 overlay 等状态，不同缓存所保存的信息和有效期并不相同。系统要求每类缓存都具有清晰的所有权、更新方式和失效条件。特别是在 buffered I/O、mmap 与 O_DIRECT 混合访问时，必须通过写回和失效操作保证页缓存与磁盘数据之间不会长期保留相互冲突的副本。

第四，在持久化路径中严格维护数据与日志的写入顺序。系统采用 JBD2 ordered 模式，普通文件数据写入其磁盘 home block，元数据修改写入日志。在提交相关元数据事务之前，与该事务相关的数据块必须已经完成必要的写回；日志的 descriptor、metadata payload 和 commit block 也必须按照规定顺序持久化。恢复阶段仅重放具有完整提交记录的事务，从而避免未完成操作在重启后被错误地视为有效状态。

第五，在并发设计中区分结构修改和纯数据访问。目录项修改、extent 调整、inode 大小变化以及块分配等操作会改变文件系统结构，需要在独占同步范围内完成。对于已经确认不会改变 extent 映射和文件大小的纯覆盖写，则可以在满足缓存一致性要求的前提下适当放宽锁粒度，减少设备 I/O 等待期间的无效串行化。所有涉及多个 inode 的操作还应遵循稳定的锁顺序，以降低死锁风险。

第六，坚持正确性优先、性能优化受约束的原则。extent 映射缓存、WrittenCoverage、批量 bio 和覆盖写快路径等机制只能在能够证明不破坏文件系统语义时启用。缓存判断应当保持保守：允许因信息不足而退化到慢路径，但不能因为错误命中而绕过必要的日志、分配或映射更新。所有性能优化都需要通过功能测试、崩溃恢复测试和实际负载测试验证，而不能仅以 benchmark 数字作为判断依据。

第七，采用可恢复的错误处理策略。文件系统不能假设磁盘数据始终合法，也不应在遇到损坏的 extent、目录项或位图时直接触发内核 panic。底层解析和修改函数应尽可能返回明确错误，上层根据错误类型终止事务、清理中间状态或进入只读、降级路径。对于 ENOSPC 等可能发生在多步骤分配过程中的错误，需要尽量通过分配记录和回滚

机制减少半完成状态。

基于上述原则，系统在逻辑上划分为应用与系统调用层、VFS 集成层、EXT4 核心语义层、JBD2 一致性层和块设备适配层。代码组织上则进一步归纳为“内核集成层”和“EXT4 核心库”两部分。前者负责将 Asterinas 的运行时机制接入文件系统，后者负责实现与具体内核环境相对独立的磁盘格式和事务语义。后续章节将在此基础上分别介绍总体架构、核心模块、数据路径以及一致性边界。

3.2 总体架构

本项目采用分层架构组织 Asterinas EXT4 文件系统。按照系统运行时的职责划分，整体架构由应用与系统调用层、VFS 集成层、EXT4 核心语义层、JBD2 一致性层和块设备适配层组成。各层分别负责用户接口、内核对象集成、磁盘格式解释、事务恢复以及设备请求提交，并通过清晰的接口连接起来。

从代码组织角度看，上述五层进一步归纳为“内核集成层”和“EXT4 核心库”两部分。内核集成层主要依赖 Asterinas 的 VFS、PageCache、Vmo、bio 和同步原语；EXT4 核心库主要负责实现 EXT4 与 JBD2 的磁盘结构和核心语义。块设备适配器连接核心库与 Asterinas block layer，使核心文件系统逻辑能够通过统一接口访问 virtio-blk 设备。

系统总体架构如图 3-1 所示。该架构展示了用户态应用、Asterinas VFS、内核集成层、EXT4 核心库、JBD2 日志子系统和底层块设备之间的主要依赖关系与数据流向。

Overall Architecture of the Asterinas EXT4 File System

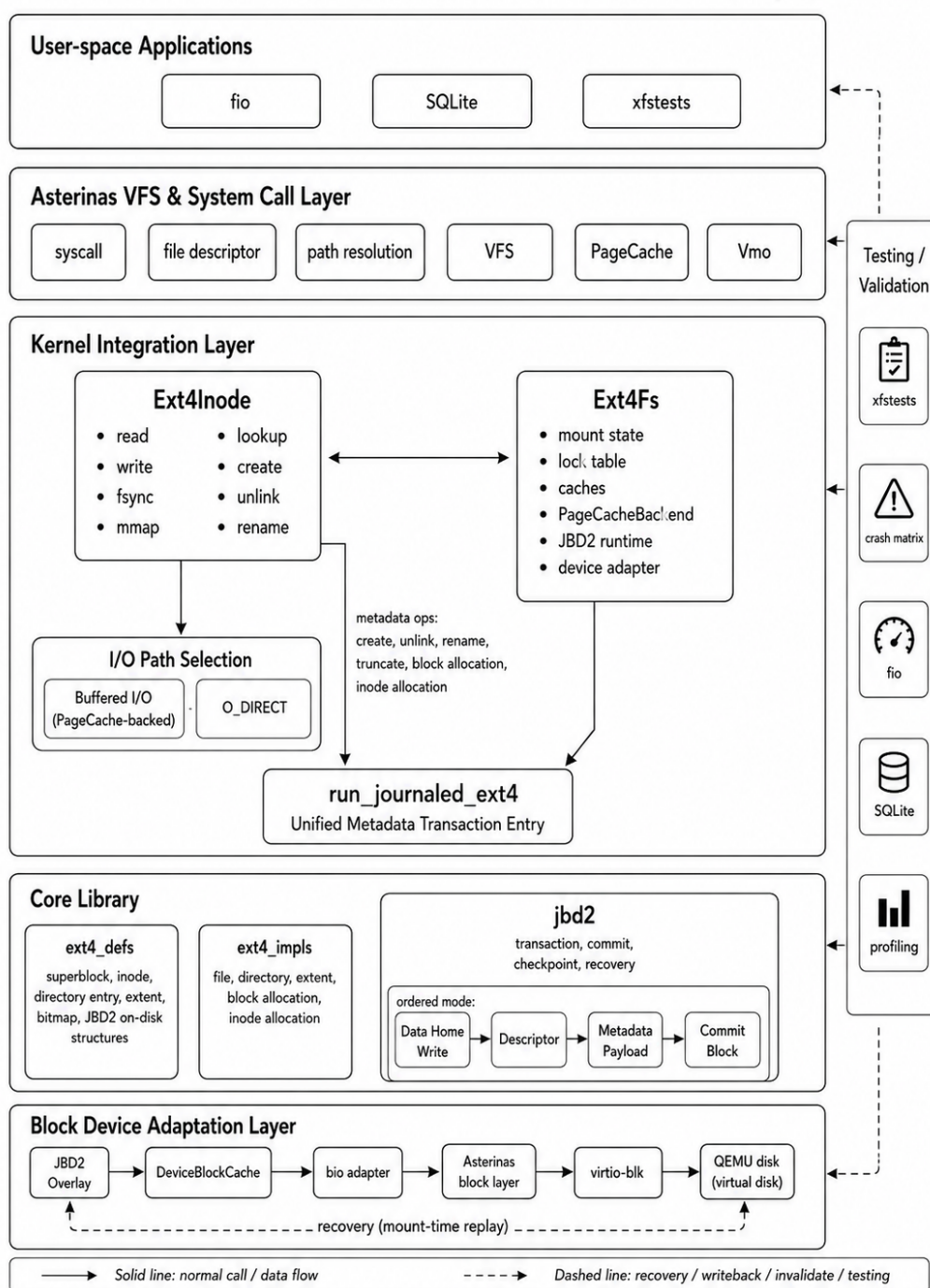


图 3-1 Asterinas EXT4 文件系统总体架构

3.3 模块划分与职责

系统按照内核集成、文件系统核心语义、日志一致性和测试验证等职责，将整体实现划分为若干相对独立的模块。各模块的主要职责与设计重点如表 3-1 所示。

表 3-1 系统模块划分与职责

模块	职责	设计重点
Ext4Inode	实现 VFS Inode trait，处理 read、write、fsync、lookup、create、unlink 和 rename 等操作入口。	根据 O_DIRECT、page_cache 和 mmap 等运行状态选择对应的数据访问路径，并维护 VFS 接口语义。
Ext4Fs	作为文件系统运行时实体，持有 Ext4 核心实例、锁表、JBD2 状态、缓存以及 PageCache 后端。	将所有元数据变更收敛到统一的 journaled chokepoint，便于执行缓存失效、错误处理和回归测试。
ext4_defs	描述 EXT4 与 JBD2 的盘上数据结构。	尽量只表达磁盘格式和字段布局，不混入 Asterinas 平台相关的运行时逻辑。
ext4_impls	实现文件、目录、Extent、块分配和 inode 分配等 EXT4 核心语义。	增强 unwritten extent、预分配、容量钳制以及文件系统错误降级等机制。
jbd2	实现事务、handle、日志提交、checkpoint 和恢复流程。	保持 ordered 模式下的数据持久化顺序，并遵守 commit block 写入前的设备屏障纪律。
测试与 benchmark	使用 xfstests、崩溃矩阵测试、fio、SQLite 和 lmbench 对系统进行测试与性能评估。	每次性能优化都必须执行正确性守底与回归验证，不能仅根据性能测试数字判断优化是否有效。

3.4 数据路径与恢复流程

运行时数据路径按照是否经过页缓存分为两类：Buffered 路径和 O_DIRECT 路径。Buffered 路径主要服务于 SQLite 等真实应用，数据先进入 PageCache，随后由 fsync 或同步路径统一写回并完成持久化；O_DIRECT 路径绕过 PageCache，直接依据 Extent 映射组织 bio 并提交到块设备。两类路径之间的一致性关键在于 buffered/direct 协同：直接写前必须先写回并失效重叠页，直接读前必须先刷脏页，避免页缓存与磁盘内容不一致。

需要单独说明的是，fsync 并不是与 Buffered、O_DIRECT 并列的第三种读写入口，

而是运行时的同步收口路径：它负责写回脏页、提交 JBD2 事务，并在最后执行设备 flush。恢复流程也不属于运行时 I/O 分流，而是在挂载或崩溃后重启时执行的 journal replay 过程。各类数据路径及其一致性要求如表 3-2 所示。

表 3-2 数据路径与恢复流程

路径	主要步骤	一致性要求
Buffered 写	判断当前写入是否属于纯覆盖写。纯覆盖写通过快速路径更新缓存页并标记为脏；文件追加、空洞写入或需要新块分配时，则进入 journaled prepare 路径。	慢路径需要先完成 Extent 映射和 inode size 更新，再将数据写入缓存；最终由 fsync 建立持久化边界。
fsync	写回文件中的脏页，提交并等待必要的 JBD2 事务，最后执行块设备 flush。	写入 commit block 之前，必须保证 descriptor 和 metadata payload 已经持久化。
O_DIRECT 写	写回并等待重叠范围内的脏页，准备 Extent 映射，组织并提交大粒度 bio，写入成功后失效对应页缓存。	不允许 PageCache 在直接写入完成后继续保留旧数据，避免后续普通读取或 mmap 访问获得过期内容。
恢复流程	扫描 journal 中已经提交的事务，按照事务顺序重放元数据块镜像，随后重置 journal 状态。	日志重放过程必须具有幂等性，即使恢复过程中再次发生崩溃，重新执行同一事务也不会破坏文件系统状态。

3.5 关键机制设计

本系统的关键机制围绕六个方面展开：挂载实例管理、VFS 入口分流、统一事务入口、页缓存后端接入、JBD2 运行时维护和块设备适配。它们共同构成了从 Asterinas VFS 到 EXT4 磁盘结构，再到 JBD2 提交与恢复的实现主干。

与 Linux 内核直接复用成熟的 address_space、writeback 和 jbd2 基础设施不同，本项目需要在 Asterinas 现有的 VFS、PageCache、Vmo、bio 和 virtio-blk 边界上重新组织这些机制，因此必须明确各对象的职责及其交互关系。各项关键机制的功能要求与本项目实现方式如表 3-3 所示。

表 3-3 系统关键机制设计

机制	优秀档要求	本项目对应实现
挂载实例管理	表示一次文件系统挂载实例，维护运行时全局状态。	持有锁表、缓存、JBD2 runtime、设备适配器以及 EXT4 核心实例。
VFS 入口分流	承接 read、write、fsync、lookup、create、unlink 和 rename 等操作入口。	在 PageCache-backed 路径与 O_DIRECT 路径之间分流，mmap 通过 PageCache 和 Vmo 接入。
统一事务入口	收敛所有涉及文件系统元数据修改的操作。	保证 create、unlink、rename、truncate 和空间分配等操作具有统一的事务边界。
缓存后端接入	将 buffered I/O 与 mmap 纳入统一缓存体系。	统一管理脏页写回、缓存失效与数据同步语义。
JBD2 运行时	维护事务、提交、checkpoint 与恢复状态。	保持 ordered 模式下的数据先行原则与日志提交顺序。
块设备适配	将文件系统请求转换为底层块设备访问。	区分 home block 视图与 JBD2 overlay 视图。

总体而言，这些关键机制将“平台对象接入”“磁盘结构解释”“事务组织”“缓存协同”和“设备提交”连接为一条完整的实现链。它们不是彼此独立的局部技巧，而是围绕统一事务入口、统一缓存语义和统一持久化边界组织起来的系统化设计。

后续的一致性边界与关键流程分析，正是在这些机制已经建立的前提下展开的。

3.6 一致性边界与关键流程

文件系统正确性的核心并不在于某个模块能否独立工作，而在于多个模块协同执行时，关键流程是否始终遵守统一的一致性约束。对本项目而言，这些约束主要体现在六个方面：元数据修改必须进入统一事务，PageCache 与 O_DIRECT 必须保持数据可见性一致，JBD2 提交必须遵守 ordered 模式下的持久化顺序，各类缓存必须具有明确的失效条件，并发访问必须区分结构性修改与纯数据覆盖，错误处理则必须保证失败能够

被局部隔离和恢复。

首先，元数据事务边界是文件系统崩溃一致性的基础。文件创建、删除、重命名、截断、块分配和 Extent 调整等操作通常不会只修改单一磁盘结构，而是可能同时涉及目录项、inode、位图、块组计数和 Extent 映射。如果这些修改分别执行并直接写入磁盘，系统一旦在操作中崩溃，就可能出现目录项已经建立但 inode 尚未初始化、数据块已经分配但 Extent 映射尚未更新，或者 inode 大小已经改变但数据块尚未准备完成等不一致状态。

为避免上述问题，系统将所有涉及元数据修改的操作统一收敛到 `run_journaled_ext4` 所提供的 `journaled transaction` 入口中。该入口负责创建或加入 JBD2 事务、记录被修改的元数据块、执行具体文件系统操作、传播执行或提交阶段产生的错误，并在事务结束后完成必要的缓存失效。

通过这一统一入口，原本分散在 `create`、`unlink`、`rename`、`truncate` 和空间分配等路径中的元数据修改被纳入同一事务边界，使恢复过程能够以完整事务为单位判断修改是否有效。其函数入口如代码 3-1 所示。

```

1 fn run_journaled_ext4<T>(  

2     &self,  

3     op: Option<JournaledOp>,  

4     apply: impl FnOnce(&Ext4) -> Result<T>,  

5 ) -> Result<T> {  

6     self.check_not_shutdown()?;  

7  

8     // 创建或加入当前 JBD2 事务  

9     // 执行文件系统元数据修改  

10    // 记录被修改的元数据块  

11    // 提交事务并完成缓存失效  

12 }

```

代码 3-1 `run_journaled_ext4` 统一事务入口

在统一元数据事务边界的基础上，`buffered I/O`、`mmap` 和 `O_DIRECT` 之间还必须遵守缓存可见性边界。`buffered` 写入和 `mmap` 修改首先体现在 `PageCache` 与 `Vmo` 中，而 `O_DIRECT` 则绕过页缓存，直接依据 Extent 映射访问块设备。因此，页缓存和磁盘可能在短时间内保存同一文件区域的不同版本，系统必须通过显式的写回和失效操作消除这种状态分叉。

对于 direct read，如果重叠范围内仍存在尚未写回的脏页，直接从设备读取就可能获得旧数据。因此，direct read 执行前必须先写回并等待重叠范围内的脏页完成持久化。对于 direct write，写入前同样需要处理重叠脏页，避免页缓存中的新数据随后覆盖直接写入结果；直接写入成功后，还必须失效对应的缓存页，防止后续普通 read 或 mmap 继续读取旧缓存。通过这套协议，buffered 路径、mmap 路径与 direct 路径虽然采用不同的数据传输方式，但最终被约束在同一个数据可见性边界内。

除运行时可见性外，持久化路径还必须满足 JBD2 ordered 模式规定的数据与日志提交顺序。普通文件数据直接写入对应的 home block，目录项、inode、位图和 Extent 等元数据修改则由 JBD2 事务组织。在事务写入 commit block 之前，与该事务相关的普通文件数据必须首先完成必要的写回，随后 journal descriptor 和 metadata payload 也必须到达稳定存储，最后才能写入表示事务已经提交完成的 commit block。

这一顺序约束使恢复逻辑能够将完整的 commit 记录作为事务是否有效的判断依据。如果系统在 commit block 写入之前崩溃，恢复阶段不会重放该事务；如果 commit block 已经持久化，则对应的 descriptor 和 metadata payload 已经满足提交顺序要求，可以被安全重放。由此可以避免尚未完成的目录项、inode 或 Extent 修改在重启后被错误地视为有效状态。因此，ordered 提交顺序既是 fsync 持久化语义成立的前提，也是 JBD2 崩溃恢复正确性的核心边界。

缓存体系内部同样需要保持清晰的状态边界。设备块缓存保存的是块设备 home 位置上的稳定数据视图，而不直接缓存叠加 JBD2 overlay 后的事务元数据状态；journal overlay 由更上层根据当前事务状态动态合成。这样可以避免尚未 checkpoint 的元数据镜像被误认为已经写回 home block，从而混淆日志状态与磁盘稳定状态。

Extent map cache 和 WrittenCoverage 等缓存则属于语义级加速信息。它们可以用于减少重复的 Extent 查询和覆盖范围判断，但不能成为绕过正确性检查的依据。对于 truncate、unlink、rename-overwrite、Extent 转换和空间重新分配等会改变文件映射含义的操作，系统必须主动清理相关缓存。即使缓存信息不足会使后续访问退回慢路径，也不能允许旧映射继续参与覆盖判断、空间分配或数据访问。换言之，每类缓存都必须具有明确的所有者、更新方式和失效时机，不能形成彼此独立且长期分叉的状态副本。

并发控制是上述一致性边界在多请求环境下的进一步延伸。目录项修改、Extent 调整、inode 大小变化和块分配等操作会改变文件系统结构，因此必须在独占同步范围内完成。对于涉及多个 inode 的 rename 等操作，还需要按照稳定顺序获取对象锁，避免不同线程以相反顺序加锁而形成死锁。

与此不同，已经确认不会改变 Extent 映射、文件大小和空间分配状态的纯覆盖

`O_DIRECT` 写，只会更新已有物理块中的文件数据。这类操作可以在满足缓存写回与失效协议的前提下使用共享锁，避免多个线程在等待设备 I/O 时被不必要地完全串行化。这里的重点并不是单纯扩大并发度，而是明确区分哪些操作会改变文件系统结构、必须独占执行，哪些操作仅更新已有数据、可以安全并行，从而保证性能优化始终处于正确性约束以内。

最后，错误处理本身也是一致性设计的一部分。文件系统不能假设磁盘上的 Extent、目录项、inode 和位图始终合法，也不能在出现损坏结构、异常映射或 `ENOSPC` 中途失败时直接触发内核 panic。底层解析与修改函数应返回明确的错误，由上层决定终止事务、清理中间状态、失效相关缓存或进入保守降级路径。

对于空间分配等包含多个步骤的操作，还应记录已经完成的分配结果，在后续步骤失败时尽可能执行回滚，减少遗留的半完成状态。即使无法完全恢复到操作前状态，也应将错误影响限制在当前事务或当前 inode 范围内，避免局部失败进一步演化为不可恢复的全局文件系统不一致。

综合来看，本项目的一致性并不是由单一模块或单一日志机制独立保证的，而是由多个相互衔接的边界共同维持：元数据修改通过统一事务入口保证原子性，`buffered`、`mmap` 与 `direct` 路径通过写回和失效协议保证数据可见性，`JBD2 ordered` 提交顺序保证持久化与恢复语义，缓存失效规则避免旧状态继续参与正确性判断，并发锁边界区分结构修改与纯覆盖访问，错误处理机制则负责将失败限制在可恢复的局部范围内。这些约束共同构成了系统从运行时访问、事务提交到崩溃恢复的完整一致性链条。

4 关键模块设计与实现

4.1 EXT4 磁盘结构与 Extent 管理

本项目使用命令 `mkfs.ext4 -b 4096` 创建测试镜像，文件系统块大小固定为 4 KiB。文件逻辑块到物理块之间的映射采用 EXT4 Extent 树实现。相比传统的直接块与多级间接块映射，Extent 可以通过逻辑起始块、物理起始块和连续块数量描述一段连续磁盘区域，因此更适合大文件与顺序写入场景，同时能够减少块映射所需的元数据数量。

EXT4 Extent 树由 Extent Header、内部节点中的 Extent Index 以及叶子节点中的 Extent Entry 共同组成。其中，Header 描述当前节点的有效条目数量、最大容量和树深度；Index 用于在非叶子节点中记录下一层 Extent 块的位置；叶子映射项则描述文件逻辑块到磁盘物理块的连续映射关系。相关盘上结构定义如代码 4-1 所示。

```
1 pub struct Ext4ExtentHeader {
```

```

2     pub magic: u16,
3     pub entries_count: u16,
4     pub max_entries_count: u16,
5     pub depth: u16,
6     pub generation: u32,
7 }
8
9 #[derive(Debug, Default, Clone, Copy)]
10 #[repr(C)]
11 pub struct Ext4ExtentIndex {
12     pub first_block: u32,
13     pub leaf_lo: u32,
14     pub leaf_hi: u16,
15     pub padding: u16,
16 }
17
18 #[derive(Debug, Default, Clone, Copy, PartialEq, Eq)]
19 #[repr(C)]
20 pub struct Ext4Extent {
21     pub first_block: u32,
22     pub block_count: u16,
23     pub start_hi: u16,
24     pub start_lo: u32,
25 }

```

代码 4-1 EXT4 Extent Header、Index 与叶子映射项定义

在基础 Extent 映射能力之上，本项目进一步补充了 unwritten extent 语义。预分配但尚未写入有效文件数据的磁盘块，会以 unwritten 状态记录在 Extent 树中。读取此类区域时，系统不能直接返回磁盘中的原始内容，而应按照文件空洞的语义向用户返回零；只有在实际写入发生后，系统才会在事务保护下将对应区域转换为 written 状态。

引入 unwritten extent 后，文件系统可以提前为顺序写入或 `fallocate` 请求批量分配连续磁盘块，在降低频繁空间分配开销的同时，避免尚未初始化的磁盘内容暴露给用户。其主要处理规则如下：

- 读取 unwritten extent 时返回全零数据，其外部语义与文件空洞一致。

- 写入 unwritten extent 时，通过 `convert_unwritten_span` 在 JBD2 事务保护下将目标范围转换为 written 状态。
- 对顺序追加写，可以在文件尾部提前预分配一段连续块，减少每写入一个 4 KiB 数据块就执行一次空间分配和 Extent 更新所产生的元数据开销。
- Extent 树解析过程中如果发现节点深度、条目数量或映射范围异常，底层函数返回 `EIO`，而不是通过 `panic` 终止整个内核运行。

上述设计使 Extent 管理不仅能够完成基本的逻辑块映射，还能够支持预分配、顺序写优化和损坏结构降级处理，为后续 buffered I/O、`O_DIRECT` 和并发写入路径提供稳定的映射基础。

4.2 JBD2 日志子系统

JBD2 日志子系统是本项目实现强一致性语义的核心模块。开源 `ext4_rs` 原有实现主要负责 EXT4 磁盘格式解析和基本文件操作，元数据修改通常直接写入其 home block，并不具备完整的事务提交与崩溃恢复能力。在此基础上，本项目新增了 `transaction`、`handle`、`commit`、`checkpoint`、`recovery` 和 `journal space` 管理等模块，使元数据修改首先进入日志，在事务提交完成后再通过 `checkpoint` 写回磁盘原始位置。

JBD2 事务从运行、提交、检查点到恢复的主要阶段如表 4-1 所示。

表 4-1 JBD2 事务处理阶段

阶段	作用	实现要点
Running	收集当前事务涉及的元数据块镜像。	<code>handle</code> 记录被修改的元数据块，对应 home block 的写回被推迟到事务提交之后。
Commit	将 <code>descriptor</code> 、 <code>metadata payload</code> 和 <code>commit block</code> 按规定格式写入 <code>journal</code> 。	在写入 <code>commit block</code> 之前执行设备 <code>sync</code> ，保证描述块和元数据镜像已经到达稳定存储。

续下页

表 4-1 JBD2 事务处理阶段（续）

阶段	作用	实现要点
Checkpoint	将已经提交事务中的元数据镜像写回对应的 home block。	根据已提交事务深度和 journal 空间水位批量推进，防止内存中的已提交事务和日志占用持续增长。
Recovery	文件系统挂载时扫描 journal，并重放已经完整提交的事务。	以完整元数据块镜像为单位进行幂等写回，未出现有效 commit block 的事务不会被重放。

事务提交时，系统首先构造并写入 journal descriptor 和 metadata payload，随后执行设备同步，确保日志描述块和元数据镜像已经到达稳定存储。在此之后，系统才构造并写入 commit block，以此标记当前事务已经完整提交。

挂载恢复时，系统只重放能够找到完整 descriptor、metadata payload 和 commit 记录的事务，从而避免未完成的元数据修改在系统重启后被错误应用。ordered 模式下的关键日志写入顺序如代码 4-2 所示。

```

1 let descriptor =
2     self.build_descriptor_block(sequence, entries.as_slice())?;
3 hook(JournalCommitWriteStage::BeforeDescriptor);
4 let mut seq_blocks: Vec<u32, &[u8]> =
5     Vec::with_capacity(entries.len() + 1);
6 seq_blocks.push((
7     descriptor_block,
8     descriptor.as_slice(),
9 ));
10 let mut data_block =
11     self.space.advance(descriptor_block, 1);
12 for entry in &entries {
13     seq_blocks.push((
14         data_block,
15         entry.data.as_slice(),
16     ));

```

```

17     data_block = self.space.advance(data_block, 1);
18 }
19 self.device.write_blocks_coalesced(
20     block_device,
21     seq_blocks.as_slice(),
22 )?;
23
24 /* Ensure descriptor and metadata payload are durable. */
25 block_device.sync()?;
26 let commit = self.build_commit_block(sequence);
27 hook(JournalCommitWriteStage::BeforeCommitBlock);
28 self.device.write_block(
29     block_device,
30     commit_block,
31     &commit,
32 )?;
33 hook(JournalCommitWriteStage::AfterCommitBlock);

```

代码 4-2 JBD2 ordered 模式下的日志提交顺序

4.3 PageCache、mmap 与 O_DIRECT

真实应用通常通过普通 read/write、mmap 和 fsync 完成文件访问与持久化，而不是完全围绕 O_DIRECT 组织 I/O。为支持这类真实负载，本项目将 Asterinas 通用 PageCache 与 Vmo 接入 EXT4 文件系统，并围绕三条相互关联的路径建立统一的数据访问语义：普通 buffered I/O 与 mmap 共享 PageCache，fsync 负责将缓存中的脏数据持久化，O_DIRECT 则绕过 PageCache 直接访问块设备，但仍需与页缓存保持一致。

Buffered I/O 与 mmap 的统一缓存路径。

普通 buffered write 和 mmap 修改最终都作用于 PageCache 中的文件页。其中，buffered write 通过 Ext4Inode 访问缓存页，mmap 则通过 Vmo 将相同的缓存页映射到进程地址空间。因此，两类访问可以共享页缓存状态、脏页跟踪和后续写回机制，避免为普通 I/O 和内存映射维护彼此独立的文件数据副本。

对于已经具有稳定 Extent 映射的文件范围，覆盖写可以直接更新对应缓存页，并将页面标记为 dirty。如果写入涉及文件追加、空洞填充或新块分配，则需要先完成必要的 Extent 映射和 inode size 更新，再将数据写入 PageCache。其中，块分配、Extent

更新和 inode 更新等元数据修改，通过相应的日志化准备过程完成。该路径的主要处理流程如图 4-1 所示。

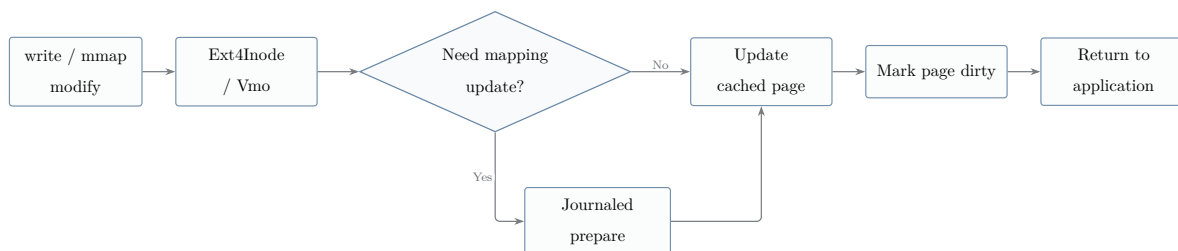


图 4-1 Buffered write 与 mmap 修改路径

普通 write 或 mmap 修改完成后，数据已经进入 PageCache，但页面仍处于 dirty 状态，因此不能认为数据已经完成持久化。后续需要通过 fsync 或同步写回流程，将缓存中的修改推进到块设备。

fsync 持久化路径。

fsync 是 buffered I/O 和 mmap 修改的持久化边界。执行 fsync 时，系统首先收集目标文件中的脏页，并将逻辑上连续的脏页组织为连续区间。这些区间随后以批量方式写回，从而减少逐页进行 Extent 映射查询以及提交大量 4 KiB bio 所产生的软件开销。

文件数据写回完成后，系统提交并等待与该文件相关的必要 JBD2 事务，随后执行设备 flush。当上述过程完成后，已经写回的页面由 dirty 状态转为 clean 状态，但页面本身继续保留在 PageCache 中。fsync 的主要处理流程如图 4-2 所示。



图 4-2 fsync 持久化路径

早期实现曾在 fsync 完成后清空整个文件缓存。该策略虽然移除了已经写回的脏页，但同时也会丢弃仍然有效的 clean 页，使 SQLite 等频繁执行提交操作的应用不断重新从设备读取工作集。

因此，当前实现不再在每次 fsync 后执行整体缓存清空，而是仅清除页面的 dirty 状态，并只在确有一致性要求时失效指定范围内的缓存页。这样既能够完成持久化，又可以保留后续访问仍然需要的缓存工作集。

O_DIRECT 与页缓存一致性。

与 buffered I/O 不同，O_DIRECT 请求根据 Extent 映射直接组织 bio，并将数据提交到块设备，不通过 PageCache 传输。但是，绕过 PageCache 并不意味着 direct I/O 可以忽略缓存中已经存在的数据状态。

对于 direct read，如果待读取范围内仍存在尚未写回的脏页，直接读取块设备可能

得到旧数据。对于 direct write，如果重叠范围内仍保留脏页，这些脏页在后续写回时还可能覆盖 direct write 的结果。因此，direct read 和 direct write 在执行前，都需要写回并等待重叠范围内的脏页。

direct write 成功后，块设备中的数据已经发生变化，而 PageCache 中可能仍保留该范围的旧副本。为防止后续普通 read 或 mmap 返回旧数据，系统还需要失效与 direct write 重叠的缓存页。该一致性处理路径如图 4-3 所示。

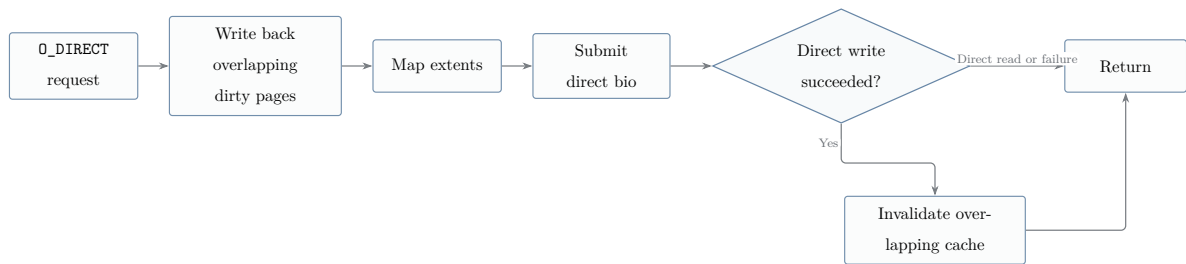


图 4-3 O_DIRECT 请求与页缓存一致性处理路径

综上，本项目围绕 PageCache 建立了统一的缓存访问与持久化语义：buffered I/O 和 mmap 负责修改共享缓存页，fsync 负责将脏页及相关事务状态持久化，O_DIRECT 则在绕过缓存传输数据的同时，通过访问前写回和写入后失效机制，维护页缓存与块设备之间的一致性。相关机制总结如表 4-2 所示。

表 4-2 PageCache、mmap 与 O_DIRECT 关键机制

机制	作用	实现说明
Buffered I/O 与 mmap 共享缓存	统一普通写入与内存映射修改的文件页状态。	buffered write 通过 Ext4Inode 更新缓存页，mmap 通过 Vmo 作用于相同页面，两者共享脏页跟踪和写回机制。
连续脏页批量写回	减少逐页映射查询和大量小 bio 带来的软件开销。	fsync 将逻辑上连续的 dirty page 合并为写回区间，再进行批量提交。
fsync 保留 clean 页	避免频繁提交场景反复重建缓存工作集。	写回完成后仅清除 dirty 状态，不再使用早期的 evict_all 清空整个缓存。

续下页

表 4-2 PageCache、mmap 与 O_DIRECT 关键机制（续）

机制	作用	实现说明
mmap writeback	支持 mmap 修改产生的脏页持久化。	mmap 与 buffered I/O 复用 PageCache 写回路径，并由 <code>pagecache_phase4</code> 等测试用例覆盖。
O_DIRECT 一致性协议	防止 direct I/O 与 PageCache 之间出现新旧数据不一致。	direct read 和 direct write 前执行重叠脏页的 write-and-wait；direct write 成功后失效重叠缓存页。

4.4 并发控制

并发控制从“能跑”到“能扩展”经历了两个阶段。第一步是将大多数文件系统操作收敛到清晰的锁获取顺序中：inode 或目录正确性锁先于 JBD2 全局运行时锁，随后再进入 Ext4 实例锁和 JBD2 runtime/journal 锁。对于涉及多个 inode 的操作，系统按照 inode 号排序获取对象锁，从而避免不同线程以相反顺序获取锁所导致的 ABBA 死锁。

第二步是将 per-inode 和 per-directory 正确性锁由普通互斥锁升级为 `RwMutex`。结构性修改仍然使用独占锁，而不会改变文件映射关系的纯数据访问可以在满足一致性条件时使用共享锁。

共享锁最关键的应用场景是 O_DIRECT 覆盖写。在 fio 使用多个 numjobs 并发写入同一个文件时，测量前文件通常已经通过 `fallocate` 建立完整映射，测量阶段的大多数写入属于纯覆盖写，不会改变 Extent 映射或文件大小。Linux EXT4 在类似场景中会使用 `shared i_rwsem`。

本项目在 `page_cache=0` 的条件下，首先获取 inode 正确性锁的共享读锁，并判断当前请求是否属于已经建立稳定映射的纯覆盖写。如果目标区间能够直接从 `Written-Coverage` 或缓存映射中确认，系统便保留共享锁，并在该锁保护下并行提交数据 bio；只有请求涉及新块分配、文件追加、空洞填充或 Extent 映射修改时，才释放共享路径并获取独占写锁。

这种设计避免了设备 I/O 等待期间不必要的 inode 级串行化，同时保证所有可能

改变文件结构的操作仍在独占锁下完成。其关键实现如代码 4-3 所示。

```

1 let inode_lock = Self::correctness_lock_for(
2     &self.inode_correctness_locks,
3     ino,
4 );
5 let mut shared_overwrite_mappings:
6     Option<Vec<SimpleBlockRange>> = None;
7 let mut shared_guard = None;
8 if !self.page_cache_enabled {
9     let guard = inode_lock.read();
10    if let Some(mappings) =
11        self.plan_direct_write_overwrite_cached(
12            ino,
13            offset,
14            write_len,
15        )?
16    {
17        shared_overwrite_mappings = Some(mappings);
18        shared_guard = Some(guard);
19    }
20 }
21
22 /*
23  * Keep the shared guard alive throughout the direct-I/O
24  * overwrite path. Fall back to the exclusive lock when
25  * the request may change the file mapping.
26  */
27 let _shared_guard = shared_guard;
28 let _excl_guard = if _shared_guard.is_none() {
29     Some(inode_lock.write())
30 } else {
31     None
32 };

```

代码 4-3 O_DIRECT 覆盖写共享锁快速路径

不同操作对应的锁模式如表 4-3 所示。

表 4-3 主要文件系统操作的锁模式

操作类型	锁模式	原因
O_DIRECT 纯覆盖读写	共享锁	不改变 Extent 映射或文件大小，设备 I/O 等待过程不应被完全串行化。
追加写、空洞写和 truncate	独占锁	可能改变 Extent 映射、空间分配状态或文件大小。
目录创建、删除和 rename	独占锁	会修改目录项以及 inode 之间的链接关系。
fsync	独占锁	需要统一推进页缓存写回和日志事务状态。

4.5 缓存体系

本项目的缓存设计并不是依靠一个“大缓存”解决所有问题，而是由多套缓存分别服务于不同的数据路径，并且所有缓存都围绕明确的更新和失效协议进行设计。缓存设计的关键并不只是提高命中率，更重要的是不能将旧数据错误地当作当前有效状态。项目在 EXT4 文件系统路径上设计了多层缓存，用于减少重复元数据读取、Extent 树遍历和数据页访问开销。

`inode_meta_cache` 用于缓存 `stat` 相关的 inode 元数据。对于 SQLite、shell 工具和测试程序中频繁出现的 `stat`、`fstat` 等调用，该缓存可以减少对 inode table 块的重复读取。为了避免返回过期元数据，系统会在 journaled mutation 完成后更新元数据缓存 generation，并根据操作类型清理受影响的 inode 缓存项。

`inode_extent_map_cache` 缓存文件逻辑块到物理块的映射关系，主要服务于 O_DIRECT 读写路径。EXT4 的 Extent 树查找需要从 inode 内嵌的 Extent root 或外部 Extent block 中逐层定位，频繁的小粒度 I/O 会放大这部分开销。通过缓存最近使用的 Extent 映射，连续读写可以复用已有结果，减少重复的 Extent 树遍历。

`DeviceBlockCache` 位于块设备适配器层，是设备 home block 的 write-through 镜像。它缓存的是设备原始位置上的块内容，而不是叠加 JBD2 overlay 后的逻辑结果。这种设计使缓存边界保持清晰：普通 home block 写入会同步更新缓存，JBD2 延迟元数据

写不会直接污染设备块缓存，读取时再由上层 journal overlay 叠加尚未 checkpoint 的日志内容。在 SQLite 测试中，该缓存对元数据块读取带来了约 98.5% 的命中率。

`WrittenCoverage` 是按 inode 维护的已写区间集合，主要用于覆盖写快速路径。对于已经通过 `fallocate` 建立完整映射，或者已经完整写入过的文件，后续覆盖写通常不会改变 Extent 映射。系统记录这类已经写入的区间，使覆盖写判定可以从 Extent 树遍历转化为一次 `BTreeMap` 前驱查询，从而降低 `O_DIRECT` 和 buffered overwrite 场景下的路径成本。

最后，`PageCache` 负责缓存普通文件数据页，支撑 buffered I/O 与 `mmap` 访问。它使普通文件读写能够在页粒度进行合并和延迟写回，并让 `mmap` 与普通 I/O 共享同一份数据缓存。对于需要绕过 `PageCache` 的 `O_DIRECT` 路径，系统会在必要时执行指定范围的脏页写回和缓存失效，避免 direct I/O 与缓存页之间出现长期不一致。

4.6 错误处理与健壮性设计

文件系统与普通应用不同，它不能假设磁盘上的结构始终合法。目录项可能已经损坏，Extent 节点可能越界，块位图可能与 inode 记录不一致，一个操作也可能在 `ENOSPC` 中途失败后留下半完成状态。开源 `ext4_rs` 在部分用户态使用场景中，遇到异常结构时会直接 panic，但这种处理方式在内核文件系统中不可接受。一次 panic 不只是终止一个进程，而可能导致整个系统停止运行。

因此，本项目在多个位置将 panic 式错误处理改为返回 `EIO` 或 `ENOSPC`，并由上层执行保守处理。Extent 树的叶节点容量、`entries_count`、索引指向的子节点以及逻辑块范围都需要显式检查；在插入或删除 Extent 时，如果发现节点状态不合理，系统不会继续假设后续内存切片必然有效，而是尽早返回错误。这种方式不一定能够修复已经损坏的文件系统，但至少可以避免一个异常节点进一步演化为内核崩溃。

另一个重要的健壮性机制是 `OperationAllocGuard`。文件系统操作通常会先分配一组磁盘块，随后再将这些块插入 Extent 树或写入相关元数据。如果操作在中途失败，系统必须知道哪些块已经完成分配，哪些块尚未真正关联到文件。否则可能出现块泄漏、重复释放，甚至“已经被 Extent 引用的块又重新回到分配器”的严重问题。Phase 6 中部分满盘压力测试暴露的问题，正是这类失败窗口被放大后的结果。

主要风险及其当前处理方式如表 4-4 所示。

表 4-4 主要健壮性风险及当前处理方式

风险类型	可能后果	当前处理
Extent 节点损坏	<code>entries_count</code> 越界，导致切片访问越界、错误映射或遍历死循环。	对节点容量和条目数量进行钳制检查，发现异常时返回 <code>EIO</code> 。
ENOSPC 中途失败	部分元数据已经写入，部分数据块未插入 Extent 或发生重复释放。	使用 <code>OperationAllocGuard</code> 跟踪分配状态；部分场景已增加防御，后续仍需补充只读降级。
缓存旧映射	将文件空洞或 <code>unwritten</code> 区间错误判断为 <code>written</code> 。	保持 <code>coverage ⊆ truth</code> ，无法确认映射有效时退回慢路径重新查询。
路径解析信息过期	目录 <code>rename</code> 后，内存中的 <code>path</code> 字段可能与真实目录结构不一致。	已列入后续改造计划，目标是以 <code>inode</code> 身份和真实目录项为准完成路径处理。

在比赛阶段，本项目尚未将每一个边界都完善到工业级文件系统的程度，但尽量遵循两项原则：第一，对已经发现的风险不进行隐瞒；第二，对于能够以较低工程成本将 panic 降级为错误返回的路径，优先补充防御处理。这样可以使 Demo 和压力测试保持相对稳定，同时也为后续进一步完善留下明确的实现入口。

4.7 运行参数与可观测性

项目在实现过程中加入了若干启动参数和环境变量，用于控制 PageCache、direct read cache、Extent map cache 以及 profiling 等行为。这样设计的原因是文件系统性能实验高度依赖测试口径。如果所有机制都固定启用，就很难判断某项优化究竟影响了哪一条路径。

例如，`fio O_DIRECT` 守底测试必须关闭 PageCache 和投机 direct read cache，而 SQLite 真实应用测试则必须启用 PageCache。通过参数化控制，可以保证不同实验针对各自目标路径，并使优化前后的数据具有可解释性。

可观测性同样十分重要。在早期 SQLite 测试中，如果只能观察总耗时，就无法判断主要瓶颈来自锁竞争、JBD2、virtio 还是 PageCache。后续实现加入了

`ext4fs.phase2_profile` 门控：该选项关闭时不会影响普通运行，开启后可以输出 buffered write fast/slow、runtime lock、block profile 和 JBD2 commit 等统计信息。这样，每次优化完成后都能够检查目标延迟桶是否真正下降。

主要参数及统计开关如表 4-5 所示。

表 4-5 主要运行参数与统计开关

参数或统计项	作用	使用场景
<code>ext4fs.page_cache</code>	控制是否启用 PageCache 数据路径。	SQLite 和 mmap 测试中开启，O_DIRECT 守底测试中关闭。
<code>ext4fs.direct_read_cache</code>	控制是否启用投机性的 direct read data cache。	为保证诚实的性能对照口径，正式测试中关闭。
<code>ext4fs.extent_map_cache</code>	控制是否启用 Extent 映射缓存。	默认开启，用于降低连续读写中的映射查询成本。
<code>ext4fs.phase2_profile</code>	开启分层性能统计和延迟归因信息。	在定位瓶颈和验证优化效果时开启。
LOG_LEVEL	控制 guest 内部日志输出级别。	性能测试时设置为 <code>error</code> ，避免串口日志干扰测试结果。

这些参数也使不同性能数据的来源更加容易解释。例如，`fio` 与 SQLite 使用不同的启动参数，是因为两者测量的是不同的数据访问路径。历史测试中 direct read 曾达到 Linux 的 127%，但该结果是在 direct read cache 开启时获得的，不属于当前关闭缓存后的真实对照口径下的测试数据，因此不作为正式性能结论引用。

5 性能优化技术研究

5.1 实验口径

性能数据必须有统一口径，否则很容易把缓存假象当成优化成果。本文采用的口径是：Asterinas 和 Linux 在同一容器、同一 QEMU/KVM、同一 virtio-blk、同一轮次中交替运行；每个 case 前清空宿主页缓存；guest 日志级别设为 `error`；自研投机 direct-read data cache 关闭；`fio` 单 job 使用 `direct=1` 的中位数。

5.2 延迟归因方法

项目后期最重要的经验是，不应先实现优化，再为优化结果寻找解释。因此，本项目首先通过分层 profiling 确定性能瓶颈所在，再根据归因结果选择对应的优化方向。

具体而言，系统使用四层 profiling，将运行耗时分别归因到文件系统路径、virtio 设备访问、锁竞争和 JBD2 日志等层次；同时使用 EXT4、EXT2 和 ramfs 三种文件系统进行对照，用于区分 Asterinas 平台本身的性能下限与 EXT4 实现引入的额外开销。在每项优化完成后，系统还会执行功能正确性测试，并记录未取得预期效果的负结果，避免在无效方向上继续投入。

本项目采用的延迟归因与验证方法如表 5-1 所示。

表 5-1 延迟归因与优化验证方法

方法	回答的问题	结论示例
四层 profiling	系统运行时间主要消耗在哪一层？	在 SQLite 初始版本中，快路径覆盖写约占 41%，慢路径分配约占 32%，commit 和 fsync 相关开销约占 24%。
三种文件系统对照	当前性能差距主要来自平台本身，还是 EXT4 文件系统实现？	EXT2 和 ramfs 分别达到 Linux 对照性能的 94.91% 和 95.87%，说明 SQLite 的主要性能瓶颈是 EXT4 的实现缺陷，而不是 Asterinas 平台的通用执行开销。
功能正确性测试	性能优化是否破坏文件系统正确性与一致性？	每次优化后均执行 crash recovery、fsync、concurrency 和 integrity_check 等测试。
负结果记录	哪些优化方向已经被实验否定，不值得继续投入？	delalloc 方案受中途写回损坏问题阻碍；size-only append 实测产生净负收益，因此最终回退。

四层 profiling 的归因关系如图 5-1 所示。该方法将一次文件系统操作的总延迟逐层拆分，从而判断主要时间消耗来自 EXT4 逻辑、锁竞争、JBD2 提交，还是底层 virtio 与块设备访问。

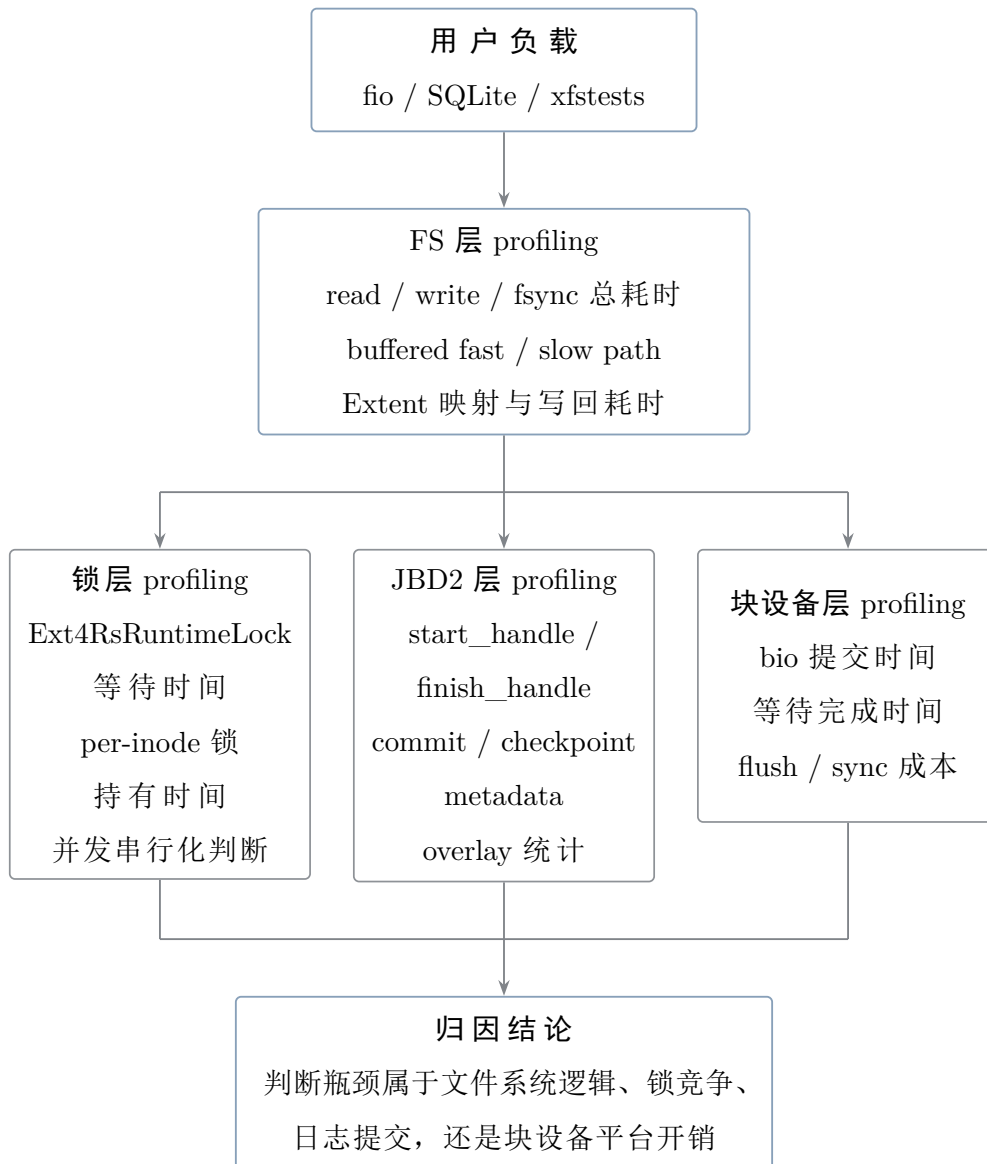


图 5-1 四层 profiling 延迟归因方法

通过上述方法，项目形成了“先定位瓶颈、再实施优化、随后执行功能正确性测试、最后记录正负结果”的完整实验流程。这种流程能够避免仅根据总耗时猜测性能问题，也使每项优化都能够与其实际攻击的延迟来源对应起来。

5.3 主要优化技术

本项目的性能优化不是一次完成的，而是围绕 profiling 暴露出的瓶颈逐步推进。

第一类优化主要面向 O_DIRECT 路径的固定开销，包括 Extent 映射缓存、inode 元数据缓存、relatime 优化、覆盖写快路径以及大块 bio 合并。优化前，小块 O_DIRECT 读写会频繁进入 Extent 树查找、inode 元数据读取和块设备提交路径，导致 4 KiB、16 KiB 等小块场景与 Linux EXT4 差距较大。引入这些优化后，重复映射查找和小块提交

开销明显下降，小块读性能从 Linux 的 16% 到 24% 提升到约 84% 到 87%。

第二类优化是 unwritten extent 与写时预分配。SQLite 在增长数据库文件时会产生大量追加写，如果每 4 KiB 写入都触发一次分配、Extent 更新和日志事务，慢路径成本会非常高。项目通过支持 unwritten extent 和预分配，将一部分频繁的小粒度分配提前合并处理，后续写入时再把对应区间转换为 written extent。该优化使 SQLite speedtest1 时间从 1772.4s 降低到 1332.2s。

第三类优化是 DeviceBlockCache。profiling 显示，SQLite 场景下大量时间消耗在元数据块的重复读取上，尤其是 inode table、目录块、bitmap 和 Extent 相关块。项目在块设备适配层加入有界 write-through 缓存，缓存 home block 内容，并在 JBD2 overlay 之下保持一致性边界。该缓存命中率约 98.5%，使 SQLite 时间从 1332.2s 大幅降低到 454.3s。

第四类优化是 WrittenCoverage 覆盖写快路径。对于已经分配并写入过的文件区间，后续覆盖写不会改变 Extent 映射，但原路径仍然可能反复查询 Extent 树。项目为每个 inode 维护已写区间集合，将覆盖写判定从 Extent 树遍历变成一次 BTreeMap 前驱查询，使覆盖写路径接近 ext2 风格的直接块写入。该优化使 SQLite 时间从 454.3s 降低到 243.9s。

最后一类优化是 O_DIRECT 共享锁。早期同一文件的并发写会被 per-inode 互斥锁串行化，即使这些写只是覆盖已分配区域，不会改变文件大小和 Extent 映射。项目将 per-inode 正确性锁升级为 RwMutex，在 page_cache=0 且命中覆盖写判定时使用共享锁，并行提交 direct I/O bio；只有需要分配或修改映射时才退回独占锁。该优化使 fio 同文件并发写在 numjobs=2 和 numjobs=4 下分别达到 Linux EXT4 的 165% 和 187%。

5.4 量化结果

经过多轮功能实现、性能归因与针对性优化，本项目在 fio、SQLite、并发写入和崩溃恢复等测试中取得了较为明显的改进。主要量化结果如表 5-2 所示。

表 5-2 系统主要量化测试结果

测试	结果	说明
fiio 0_DIRECT 读	4K/16K/64K/256K/1M 块大小下，相对 Linux EXT4 的性能分别为 101.3%/97.0%/92.9%/96.0%/92.0%。	顺序读取性能从小块到大块都接近甚至优于 Linux EXT4。
fiio 0_DIRECT 写	4K/16K/64K/256K/1M 块大小下，相对 Linux EXT4 的性能分别为 97.3%/91.0%/91.7%/113.1%/108.9%。	顺序写入性能小块接近，大块优于 Linux EXT4。
SQLite speedtest1	墙钟时间由 2010.7s 降低至 234.9s。	整体性能提升约 8.6 倍，全程 integrity_check 通过。
并发同文件写	numjobs=2 时达到 6024 MB/s，numjobs=4 时达到 5139 MB/s。	相对 Linux EXT4 的性能分别达到 165% 和 187%。
崩溃恢复	JBD2 crash matrix 18/18 PASS，host-crash fsync 4/4 PASS。	表明性能优化后仍保持了崩溃恢复与 fsync 持久化语义的正确性。

从结果可以看出，系统在合成顺序 I/O 和并发覆盖写场景下已经接近或超过 Linux EXT4，SQLite 真实应用性能也有了很大提升。与此同时，所有主要性能优化均通过崩溃恢复、fsync 和数据完整性测试验证正确性，说明当前性能提升没有以破坏文件系统一致性为代价。

表 5-2 中的主要性能结果进一步可视化为图 5-2、图 5-3、图 5-4 和图 5-5。

图 5-2 给出了不同块大小下 fiio 0_DIRECT 顺序读写相对于 Linux EXT4 的性能比例。可以看到，顺序读在各块大小下均稳定在 Linux EXT4 的 92%–101% 区间；顺序写在小块（4K–64K）下接近 Linux，而在 256K 和 1M 大块下分别达到 113.1% 和 108.9%，反超 Linux EXT4。

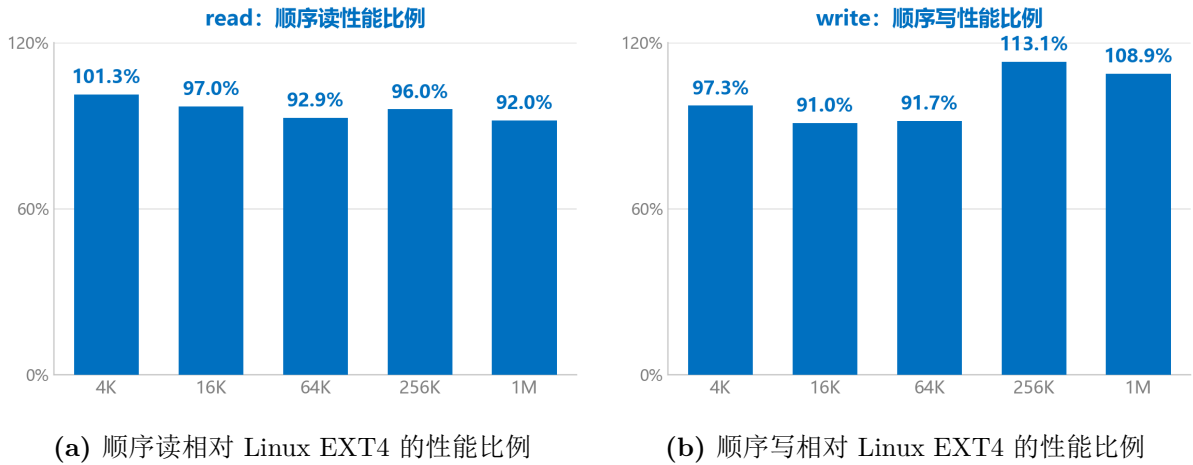


图 5-2 fio O_DIRECT 顺序读写相对 Linux EXT4 的性能比例

图 5-3 进一步给出了顺序读写的绝对吞吐对比。整体趋势上，Asterinas EXT4 与 Linux EXT4 高度贴合；在写入路径的 256K 和 1M 大块下，Asterinas EXT4 的吞吐（716.0 MB/s、742.3 MB/s）已明显高于 Linux EXT4（633.0 MB/s、681.7 MB/s）。

图 5-4 展示了引入 O_DIRECT 覆盖写共享锁快速路径后，同一文件并发写入的性能结果。在 numjobs=2 和 numjobs=4 时，系统分别达到 Linux EXT4 的 165% 和 187%，说明纯覆盖写路径中的共享锁设计有效减少了设备 I/O 等待期间的不必要串行化。

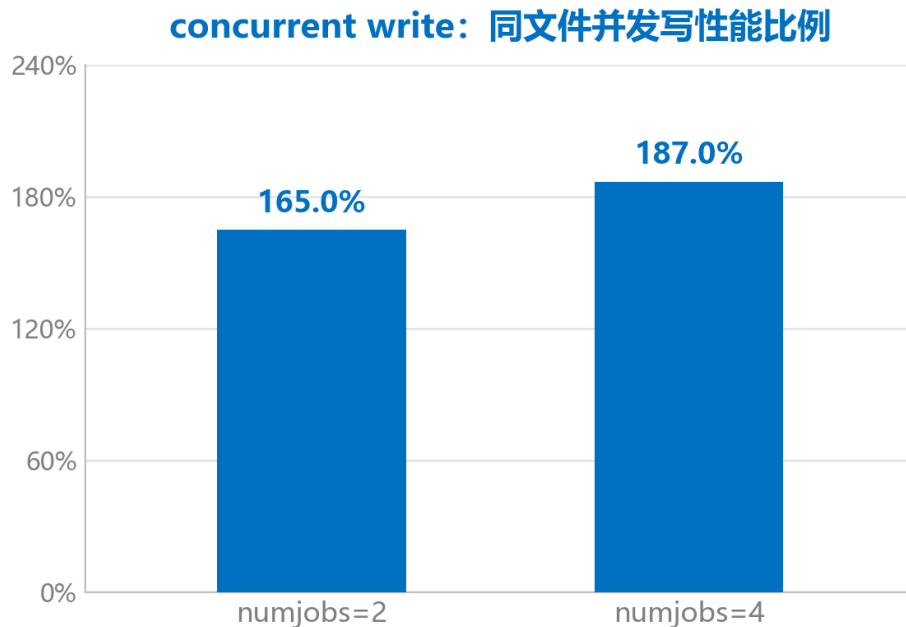
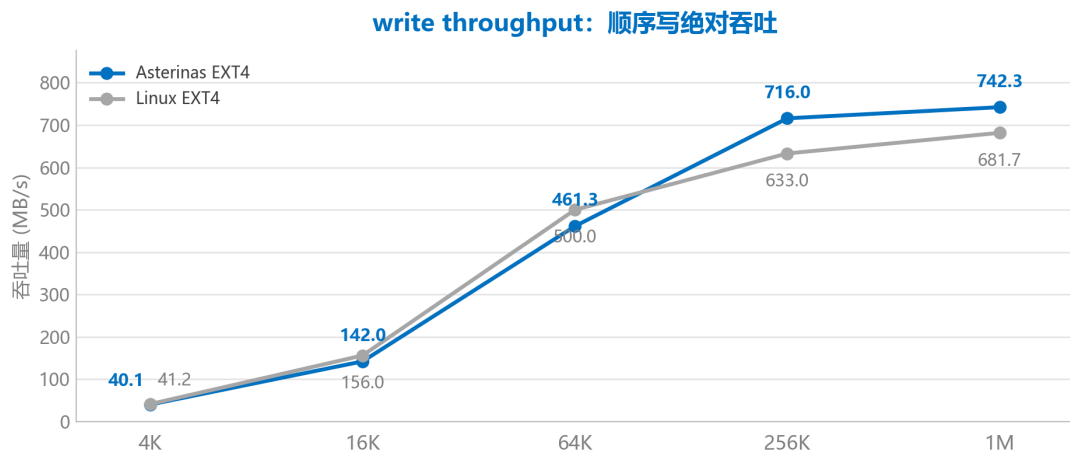


图 5-4 共享锁优化后的 fio 同文件并发写性能

图 5-5 展示了 SQLite speedtest1 在优化前后的性能变化。经过上述优化，墙钟时间由约 2010.7s 降低至 234.9s，整体提升约 8.6 倍，且各轮测试后的 integrity_check 均通过。



(a) 顺序读绝对吞吐: Asterinas 与 Linux EXT4 对比



(b) 顺序写绝对吞吐: Asterinas 与 Linux EXT4 对比

图 5-3 fio 0_DIRECT 顺序读写绝对吞吐对比

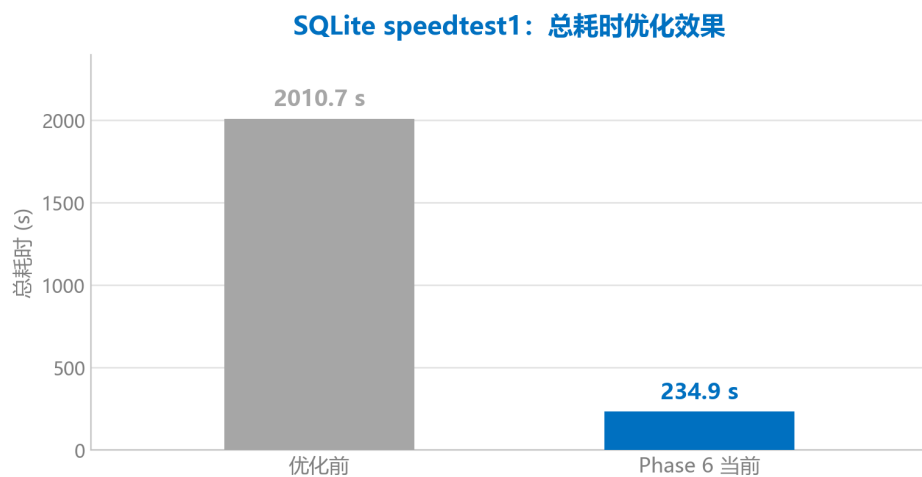


图 5-5 SQLite speedtest1 优化前后性能对比

综合来看, 当前优化已经有效缩短了 direct I/O 与并发写路径上的差距, 但数据库类负载暴露出的事务提交、元数据更新和小块追加写成本仍然较高。后续优化需要继续

围绕真实应用写路径展开，并在现有崩溃恢复、fsync 语义和数据完整性验证测试框架下逐步推进。

5.5 剩余差距分析

SQLite 即使经过大量优化后仍明显落后于预期，这一现象需要主动给出解释。本项目通过三个文件系统对照实验发现，Asterinas 原生 ext2 在同一平台上可以达到 Linux 对照性能的 94.91%，ramfs 可以达到 95.87%，说明平台本身并不是 SQLite 延迟很高达不到 Linux 性能的主要原因。剩余差距主要来自当前“写时同步转换”的实现方式：每一次 4 KiB 新分配写都需要承担零填充、handle/overlay 维护以及 unwritten 转换等额外成本。

Linux EXT4 的优势之一是 delalloc，它能够将大量 4 KiB 页面产生的元数据成本摊薄到一个 extent 上。本项目曾尝试沿 delalloc 方向推进，但完整的 delalloc 机制需要途中写回与脏页节流配合，而 Asterinas 当前 EXT4 实现中的非 fsync 点写回会触发 SQLite 数据损坏。出于对正确性的考虑，本项目选择暂缓 delalloc 的完整实现。

5.6 优化过程中的负结果

性能优化过程中，本项目同样记录了未能取得预期效果的负结果。起初，团队认为负结果不适合写入比赛文档，因为这类记录容易被理解为对失误的承认；但进一步思考后发现，如果报告只呈现成功的几个步骤，反而无法真实反映系统类项目的实际研究过程。尤其是文件系统优化，许多方向从直觉上看都具有合理性，只有在完整工作负载和守底测试之后，才能判断其是否真正成立。

第一个负结果是早期的批量写回优化，仅取得较小收益。原因并非批量写回方向本身有误，而是当时只合并了部分 bio，没有真正消除逐页 handle，也没有事先确认当时最大的瓶颈所在。后续 profiling 显示，SQLite 场景下最大的单一开销并非简单的 bio 数量，而是每次写入仍需重新读取 Extent 树块并在持锁状态下完成映射查询。这一教训直接促成了此后“先找性能瓶颈在哪、再优化”的工作纪律。

第二个负结果是 delalloc。理论上，delalloc 是最接近 Linux EXT4 的优化方向，但实测中发现，完整 delalloc 所需的脏页节流必须配合途中写回，而途中写回在当前平台会导致 SQLite 数据损坏。项目组通过隔离实验证明，该损坏并非源于 delalloc 实现中的某一行代码，而是非 fsync 点写回路径本身存在不安全问题。这一结果促使我们合理调整项目方案。

第三个负结果是 size-only append 方案。该实验试图推迟 unwritten 转换的时机，

以降低追加写的开销，但三组配置的实测结果均慢于基线数据，并出现了未能解释的读路径性能回退。按照既定原则，净负收益的优化不能因为方案“看起来更先进”而被保留，因此该方案最终被回退，仅保留相应 patch 与分析记录。

这些负结果同样是本项目方法论的重要组成部分。它们表明，项目并非为了凑出更多优化点而堆叠功能，而是在数据不支持某一方向时愿意停下并如实记录。对于学术型赛题而言，这部分内容也展示了项目组在研究过程中的判断依据与取舍逻辑。

5.7 Rust 与 C 版 EXT4 的差异思考

Rust 版 EXT4 与 Linux C 版 EXT4 之间的差异，不能简单理解为“Rust 更安全，所以一定更好”，也不能理解为“C 更接近底层，所以一定更快”。Linux EXT4 是多年工程积累的结果，大量机制已经围绕 Linux VFS、block layer、writeback 和 page cache 进行了深度优化。Rust 版文件系统在内存安全和类型表达方面具有优势，但在平台基础设施、生态成熟度和历史优化积累方面并不占优。

在本项目中，Rust 的优势主要体现在以下几个方面。首先，许多运行时状态可以通过类型系统和所有权关系进行更加明确的表达，例如 block range、journal plan、operation guard 和 cache state 等，从而减少手工管理对象生命周期和共享状态所带来的风险。

其次，错误可以通过 `Result` 在模块接口之间显式传播。相比在异常路径中直接触发 panic，这种方式更容易形成统一的错误处理和保守降级策略，也能够避免局部文件系统错误进一步演化为整个内核崩溃。

再次，Rust 有助于建立较为清晰的模块边界。本项目将平台相关的内核集成逻辑与 EXT4 核心磁盘语义分离，核心库和集成层之间通过明确接口交互。这种组织方式不仅有利于代码维护和功能测试，也使系统设计与实现过程更容易进行文档化说明。

但是，Rust 并不能代替文件系统设计本身。JBD2 的提交屏障、revoke 语义、fsync 与 PageCache 的协同，以及 `O_DIRECT` 与 buffered I/O 之间的一致性，本质上都属于文件系统语义和持久化协议问题，不会因为实现语言发生变化而自动得到解决。

同样，SQLite 当前仍然存在的性能差距，也不能简单归因于 Rust 语言的执行效率。其主要原因在于当前系统尚未具备与 Linux EXT4 同等成熟的 writeback、delalloc 和空间分配基础设施。这一结论对本项目十分重要：既需要说明 RustOS 在内存安全、模块边界和生态建设方面的价值，也不能将所有性能与一致性问题简单归因于编程语言。

Linux C EXT4 与本项目 Rust EXT4 的主要差异如表 5-3 所示。

表 5-3 Linux C EXT4 与本项目 Rust EXT4 的差异

维度	Linux C EXT4	本项目 Rust EXT4	结论
内存安全	主要依赖开发者约束、代码审查以及 Linux 内核中的防御机制。	Rust 类型系统、所有权和生命周期机制能够提供更多静态安全保证。	Rust 在内存安全方面具有优势，但 unsafe 代码和磁盘格式解析仍然需要谨慎处理。
性能积累	delalloc、writeback、mballoc 和 Extent status tree 等机制已经较为成熟。	已实现部分缓存、预分配和映射优化，但 delalloc 仍受到平台写回能力的限制。	Linux C 版具有更深的工程积累，Rust 实现不能仅依靠语言优势简单追平。
崩溃恢复	JBD2 机制较为完整，包括事务提交、checkpoint、recovery 和 revoke。	JBD2 主体流程已经完成，但 revoke 记录的写入路径仍待补充。	已具备主体崩溃恢复能力，但尚未闭合的边界需要在文档中如实说明。
平台适配	与 Linux VFS、page cache、writeback 和 block layer 深度绑定。	面向 Asterinas 的 OSTD、VFS、PageCache、Vmo、bio 和 virtio-blk 接口进行适配。	本项目的主要价值之一，是补齐 RustOS 环境中的主流磁盘文件系统能力。

综合来看，Rust 为文件系统实现提供了更强的静态约束、更清晰的错误传播方式和更明确的模块边界，但这些优势并不会自动转化为完整的崩溃一致性和更高的 I/O 性能。文件系统最终仍然需要依靠正确的事务协议、缓存一致性设计、空间分配策略和写回基础设施。

因此，本项目并不试图证明 Rust EXT4 在所有方面优于 Linux C EXT4，而是希望说明：在 RustOS 和 framekernel 环境下，可以利用 Rust 的安全与工程表达能力实现一个具有完整主体功能的 EXT4 文件系统，同时也应客观认识与成熟 Linux EXT4 在基础设施和优化积累方面的差距。

6 系统测试与验证

6.1 测试环境

为保证测试结果具有可复现性，本项目统一在 QEMU/KVM 和 virtio-blk 环境下运行 Asterinas EXT4，并使用相同环境中的 Linux EXT4 作为对照。具体测试环境如表 6-1 所示。

表 6-1 系统测试环境

项目	配置
宿主系统	Ubuntu 24.04.1, Linux 6.8.0-41。
虚拟化环境	QEMU/KVM, 块设备采用 virtio-blk。
容器镜像	asterinas/asterinas:0.17.0-20260227。
虚拟机配置	8 GiB 内存, 单处理器, 启用 KVM 加速。
磁盘格式	使用 <code>mkfs.ext4 -b 4096</code> 创建 EXT4 测试镜像, 文件系统块大小为 4 KiB。
对照系统	相同虚拟化与块设备环境下的 Linux EXT4。

6.2 xfstests 功能正确性测试

本项目按照功能开发阶段组织了多组 xfstests 测试集合，分别覆盖基础文件与目录操作、PageCache、JBD2、fsync 持久化语义以及并发访问等关键路径。各测试集合的结果如表 6-2 所示。

表 6-2 xfstests 功能正确性测试结果

套件	结果	覆盖点
phase3_base_gu ard	10 PASS: generic/006、013、028、047、051、052、054、068、083、084; 0 FAIL; 6 NOTRUN。	基础文件、目录操作与 fsync 功能基线。
phase4_good	12 PASS: generic/001、006、013、028、035、047、051、052、054、068、083、084; 0 FAIL; 6 NOTRUN。	PageCache 缓冲 I/O 基础功能。
phase6_good	25 PASS: generic/001、003、006、007、011、013、014、015、027、028、035、047、051、052、054、068、076、083、084、087、100、124、126、184、192; 0 FAIL; 1 NOTRUN。	Phase 6 相关功能守底测试。
pagecache_phas e4	9 PASS: generic/091、130、133、247、263、412、418、469、749; 0 FAIL; 4 NOTRUN。	buffered/direct 一致性、mmap 与脏页 fsync。
jbd_phase1	6 PASS: generic/068、076、083、192、ext4/021、ext4/045; 0 FAIL; 6 NOTRUN。	JBD2 事务、日志刷盘与崩溃恢复基线。
jbd_phase3_fsy nc_durability	11 PASS: generic/043、044、045、046、047、049、052、054、055、388、392; 0 FAIL; 1 NOTRUN。	fsync、fdatasync 和 flush 持久化语义，覆盖 inode size、Extent、日志重放、恢复幂等性及 fsync/fdatasync 边界。
concurrency	10 PASS: generic/013、068、076、083、476、269、051、054、055、388; 0 FAIL。	fsstress、多进程并发及并发直接 I/O。

从上述结果可以看出，当前纳入正式测试范围的核心功能集合均保持 0 FAIL。不同测试集合分别覆盖文件系统开发中的不同阶段，使 PageCache、JBD2 和并发优化能够在已有正确性基线之上逐步推进。

6.3 xfstests 兼容性测试

本项目将 xfstests 作为功能正确性验证的主要外部标准。测试入口位于：

```
test/initramfs/src/syscall/xfstests/testcases/
```

不同开发阶段通过 list 文件组织测试用例，运行日志以 detailed results 中的 PASS、FAIL 和 NOTRUN 作为判定依据。其中，PASS 表示该 xfstests case 在 Asterinas EXT4 上执行完成并返回 `rc=0`；FAIL 表示文件系统行为与预期不一致；NOTRUN 表示由于测试环境不满足要求，或者当前能力尚未实现而被静态跳过。

NOTRUN 的常见原因包括缺少 AIO 辅助程序、缺少 debugfs 或 device-mapper、暂不支持 hardlink，以及 scratch 空间不足等。因此，NOTRUN 与功能执行失败需要分别统计，不能直接将其视为 FAIL。

需要说明的是，official xfstests 全量测试目前仍在持续收口。当前未纳入通过结论的部分主要来自以下三类原因。

第一，Asterinas 当前尚未提供 device-mapper、debugfs 和 loopback 等测试环境能力，因此依赖 dm-log-writes、dm-flakey 以及部分 debugfs 工具的用例无法直接运行。

第二，hardlink、symlink 以及部分 xattr/attr 能力尚未完整实现，相应测试会被静态跳过或无法满足测试前置条件。

第三，在 full run 中仍有少量用例可能触发堆分配错误，或者暴露 PageCache 更底层的异常路径。

目前，本项目对于赛题核心路径相关的 xfstests 子集已经做到 0 FAIL，并在报告中给出明确的通过用例与覆盖范围；official xfstests 全量通过率达到 95% 仍作为后续完善目标。

6.4 崩溃一致性测试

Asterinas 当前不提供 device-mapper，因此无法直接使用 xfstests 中依赖 dm-log-writes 和 dm-flakey 的崩溃注入方式。为验证 JBD2 提交与恢复语义，本项目设计了一些崩溃注入场景，在 JBD2 提交记录写入前后、checkpoint 前后等关键语义点主动终止虚拟机，随后重新启动 Asterinas、挂载 EXT4，并通过文件内容、CRC、目录结构和 fsync 持久化结果进行校验。

崩溃一致性测试结果如表 6-3 所示。

表 6-3 崩溃一致性测试结果

测试	结果
JBD2 崩溃恢复矩阵 (Phase 1)	9/9 PASS
JBD2 崩溃恢复矩阵 (Phase 2/4)	18/18 PASS
host-crash fsync 持久化测试	4/4 PASS
SQLite integrity_check	每轮性能测试后均 PASS

Crash Matrix 的完整测试流程如图 6-1 所示。

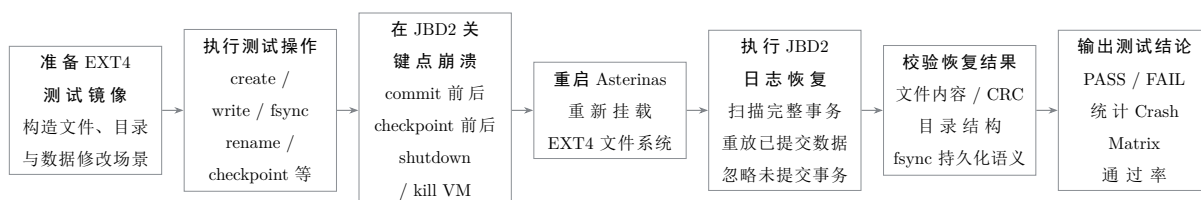


图 6-1 Crash Matrix 崩溃一致性测试流程

测试首先准备 EXT4 镜像，并构造文件、目录或元数据修改场景；随后执行 create、write、fsync、rename 或 checkpoint 等目标操作，并在 JBD2 commit、checkpoint 前后等关键位置触发虚拟机退出。系统重新启动后执行日志扫描与事务重放，最后通过文件内容、CRC、目录结构及 fsync 持久化语义判断恢复结果是否正确。

与随机终止进程相比，这种测试方式能够更准确地覆盖 JBD2 事务边界。如果 descriptor、metadata payload 和 commit block 的持久化顺序不正确，或者恢复逻辑错误地重放了未提交事务，相应测试就会直接失败。

6.5 并发正确性测试

并发正确性不能只依据 fio 带宽进行判断。高吞吐只能说明并发请求得到了执行，不能证明文件内容没有发生错乱、丢失或相互覆盖。因此，本项目使用了两层并发正确性证据。

第一层是测试程序 `phase2_concurrency.c`。多个 worker 对多个文件执行具有确定性数据模式的并发读写，测试完成后由主进程逐一检查文件大小与内容 hash。第二层是 `xfstests concurrency` 套件，使用 `fsstress` 等随机压力负载补充更多并发操作组合。两类测试均通过，说明共享锁改造没有引入已知的数据错乱或数据丢失问题。

并发写入与确定性 hash 校验的关键实现如代码 6-1 所示。

```

1 static void case_multi_file_write_verify(void)
2 {
3     char case_dir[PATH_MAX];
4     ensure_case_dir(case_dir, sizeof(case_dir));
5     spawn_workers(worker_multi_file_write, case_dir, "multi_file_write")
6     ;
7     for (unsigned int worker = 0; worker < g_workers; worker++) {
8         char path[PATH_MAX];
9         char name[64];
10        snprintf(name, sizeof(name), "worker_%02u.dat", worker);
11        path_join(path, sizeof(path), case_dir, name);
12        verify_file_hash(
13            path,
14            expected_hash(worker, g_rounds, IO_BLOCK, 0x11),
15            (off_t)g_rounds * IO_BLOCK
16        );
17    }
18 }

```

代码 6-1 并发写入测试与确定性 hash 校验逻辑

该代码片段体现了并发测试的基本验证方式。测试用例首先创建独立的测试目录，并通过 `spawn_workers` 启动多个 worker 执行并发写入。所有 worker 结束后，主进程按照 worker 编号逐一定位输出文件，再使用 `verify_file_hash` 对实际文件内容进行校验。

期望 hash 由 `expected_hash` 根据确定性数据模式计算得到，期望文件大小为：

$$g_rounds \times IO_BLOCK.$$

因此，该测试并不是简单判断并发任务是否退出成功，而是进一步验证并发执行后每个文件的数据内容和文件大小是否符合预期。这种确定性验证方式可以直接裁决并发写入过程中是否出现错写、漏写或覆盖错误。

6.6 性能测试

本项目的性能测试同时包含合成负载和真实应用负载，分别用于衡量基础 I/O 吞吐、不同参数下的性能变化以及数据库工作负载中的综合表现。主要测试类型如表 6-4 所示。

表 6-4 性能测试类别

类别	工具	用途
合成顺序读写	fio, direct=1	评估 O_DIRECT 基础吞吐，验证不同块大小下与 Linux EXT4 的性能差距。
参数广度扫描	fio sweep, 共 96 个 case	观察块大小、numjobs、fsync、page_cache 等维度对性能的影响。
真实应用	SQLite speedtest1	评估文件系统在高频小事务和数据库写入负载下的综合表现。

fio 顺序 I/O 用于观察基础数据路径，并验证 Extent 映射、bio 提交和 virtio-blk 等路径的吞吐能力。fio 参数 sweep 则用于研究块大小、并发度、同步方式和缓存配置对文件系统性能的影响。

SQLite speedtest1 代表更接近真实应用的负载。该测试同时涉及 buffered I/O、PageCache、文件扩展、fsync、JBD2 提交和元数据读取，因此能够暴露单纯顺序吞吐测试无法体现的综合开销。每轮 SQLite 测试后均执行 integrity_check，避免将数据损坏情况下得到的较短运行时间误认为有效性能提升。

6.7 测试用例设计说明

本项目的测试设计并不是简单罗列能够运行的脚本，而是按照文件系统的主要风险组织测试覆盖。

基础 POSIX 操作主要由各种 xfstests 测试来覆盖；崩溃一致性由 Crash Matrix 与 host-crash fsync 覆盖；并发正确性由确定性 hash 校验与 xfstests concurrency 覆盖；性能则由 fio 和 SQLite 共同验证。不同类型的测试互相补充，避免因为单一测试通过而对系统正确性作出过度推断。

Crash Matrix 的设计重点是将崩溃注入放在 JBD2 的关键语义点，而不是随机终止进程。例如，descriptor 和 metadata payload 写入之后、commit block 写入之前，以

及 checkpoint 前后，都是文件系统恢复语义最关键的位置。如果 commit block 之前的 payload 没有正确持久化，或者 recovery 对事务提交边界识别错误，这些场景能够更直接地暴露问题。

并发测试采用两层结构也有明确原因。自研 `phase2_concurrency.c` 的优点是期望结果确定，可以通过文件大小和 hash 精确判断数据是否错乱；其缺点是负载形态相对有限。xfstests concurrency 的优势是 fsstress 的操作组合更加随机，更接近真实压力环境，但失败后的定位成本较高。二者结合可以同时获得可裁决性和更广的操作覆盖。

性能测试则分为“功能正确性测试”和“研究测试”。功能正确性测试关注优化是否导致已有性能或正确性回退，例如 `O_DIRECT` 五档块大小、lmbench 和 SQLite `integrity_check`。研究测试关注系统为什么慢以及优化应攻击哪个瓶颈，例如 fio 参数 `sweep` 和 SQLite 三个文件系统对照。将二者分开，可以避免将研究阶段的临时测试数据误当作最终性能成绩。

测试层次与覆盖关系如表 6-5 所示。

表 6-5 测试层次与风险覆盖关系

测试层次	代表用例	主要风险	判定方式
基础功能	各种 xfstests	POSIX 接口行为错误，文件和目录操作结果不符合预期。	xfstests 返回 <code>rc=0</code> ，且 FAIL 数量为 0。
崩溃恢复	Crash Matrix、host-crash fsync	JBD2 提交、checkpoint 或恢复边界处理错误。	重新挂载后 CRC、文件内容和目录结构保持一致。
并发正确性	<code>phase2_concurrency.c</code> 、xfstests concurrency	并发读写发生数据错乱、内容丢失、死锁或 panic。	确定性 hash 校验通过，测试套件全部 PASS。
页缓存一致性	<code>pagecache_phase4</code>	<code>O_DIRECT</code> 与 buffered I/O、mmap 之间出现数据不可见或缓存旧数据。	通过不同路径读回的数据内容与预期一致。

续下页

表 6-5 测试层次与风险覆盖关系（续）

测试层次	代表用例	主要风险	判定方式
性能	fio、SQLite、lmbench	吞吐不足、延迟过高，或者优化导致历史性能回退。	与同口径 Linux EXT4 及项目历史基线进行对照。

整个测试体系的覆盖关系如图 6-2 所示。

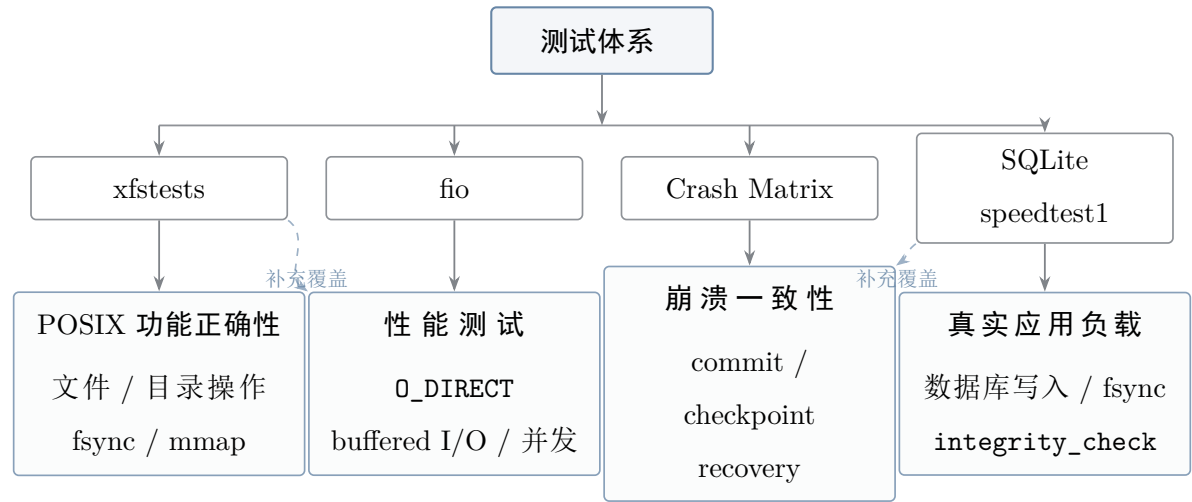


图 6-2 系统测试体系覆盖关系

该测试体系以 xfstests、fio、Crash Matrix 和 SQLite speedtest1 为主要工具，分别覆盖 POSIX 功能正确性、基础性能、崩溃一致性和真实应用负载。不同工具之间存在一定交叉覆盖，例如 xfstests 同时参与功能与并发守底，SQLite 同时验证实际性能和数据完整性。

6.8 测试结论分析

从当前测试结果来看，本项目已经基本满足优秀档实现完整度中的大部分核心要求。JBD2 的事务、提交、checkpoint 和恢复路径已经通过多场景崩溃测试；多文件并发读写具有确定性校验测试和 xfstests concurrency 两层证据；纳入正式测试范围的核心功能套件均保持 0 FAIL。

对于比赛初赛而言，这些结果比单纯展示一个能够运行的 Demo 更重要，因为它们不仅验证了系统在正常执行路径中的功能，也验证了系统在并发访问、同步持久化和异常崩溃场景下的行为。

性能测试结果需要按照数据路径分别分析。在 O_DIRECT 顺序读写场景中，系统从

小块到大块的读写带宽均十分接近甚至反超 Linux EXT4，基本满足赛题的性能要求，接下来还可以进一步探索具体的优化策略。

在 buffered I/O 和 SQLite 方向，系统性能已经获得显著提升，但仍未达到 Linux EXT4 的 90%。这部分差距主要来自当前写回模型以及 delalloc、后台 writeback 和空间预留机制的不完整，而不是单一函数或单一锁造成的局部问题。项目组正在与赛题组及 Asterinas 维护者沟通相关基础设施问题，并探索适合当前 framekernel 架构的解决方案。

测试同时暴露了后续需要继续完善的方向。首先，revoke 写入路径尚未闭合，需要增加针对元数据块释放和重新复用场景的崩溃测试。其次，ENOSPC 中途失败后的半完成状态仍需要通过更严格的回滚和 e2fsck 门控进行验证。最后，official xfstests 全量测试仍需继续分类收口，并补充 hardlink、symlink、xattr 和部分测试环境能力。

总体而言，当前测试结果足以支撑本项目初赛阶段的主要完成度结论，但仍不能据此宣称系统已经达到生产级 Linux EXT4 的完整性与成熟度。本项目当前完成的是一个初步具备 EXT4 主体功能、JBD2 崩溃恢复能力、真实应用支持和系统化测试证据的 RustOS 文件系统实现，但仍有许多尚未完善的地方和性能优化空间。

7 项目创新点

本项目的创新点并不局限于一两个局部代码技巧，而是形成了一条围绕 RustOS 文件系统实现、崩溃一致性和性能优化的完整技术路线。结合系统设计、实现过程和实验结果，本项目具有代表性的创新点主要包括以下六个方面。

1. 在 Asterinas 非特权 framekernel 环境中构建完整的 EXT4/JBD2 文件系统。

本项目将开源 `ext4_rs` 从面向用户态或磁盘格式解析的基础库，扩展为能够在 Asterinas 内核中运行的文件系统，补充 VFS 集成、PageCache、块设备适配、并发控制、JBD2 事务提交和崩溃恢复等机制。

2. 设计位于 JBD2 overlay 之下的 DeviceBlockCache。

该缓存采用 write-through 的设备 home block 镜像，用于统一承接 inode 块、Extent 树块、块组描述符和位图等热点元数据读取。它不会直接缓存叠加 JBD2 overlay 后的逻辑结果，因而能够在提高读取命中率的同时保持日志语义清晰。在 SQLite 测试中，该缓存对相关元数据块读取的命中率约为 98.5%。

3. 实现 unwritten extent 语义与写时预分配。

系统可以提前批量分配连续磁盘块，并将尚未写入有效数据的区域标记为 unwrit-

ten。读路径对该区域返回零，写路径则在 JBD2 事务保护下将其转换为 written，从而兼顾空间预分配性能与未初始化数据隔离。

4. 设计 WrittenCoverage 已写区间集合。

WrittenCoverage 以 inode 为单位记录已经确认写入并具有稳定映射的区间，使覆盖写快速路径能够从重复遍历 Extent 树，转化为一次基于 BTreeMap 的前驱查询。其核心不变量为 $\text{coverage} \subseteq \text{truth}$ ，即缓存可以少记，但不能将空洞或 unwritten 区间错误记录为 written。

5. 对齐 Linux shared i_rwsem 的并发覆盖写语义。

本项目将 per-inode 正确性锁由互斥锁改造为 RwMutex，对不会改变 Extent 映射和文件大小的 O_DIRECT 纯覆盖写使用共享锁，只有在追加、空洞填充或空间分配时才进入独占锁路径。该设计显著缓解了同文件并发写中的锁串行瓶颈。

6. 形成可复用的系统性能研究方法。

项目采用“四层 profiling、三个文件系统对照、优化后的正确性测试”相结合的方法，将性能差距拆分为文件系统逻辑、锁竞争、JBD2 提交、virtio 和平台基础设施等可解释成分，并对无效或存在正确性风险的优化方向进行负结果记录。

总体而言，本项目的创新性既体现在具体机制上，也体现在面向 RustOS 文件系统的工程方法上。这些设计不是彼此独立的优化补丁，而是围绕事务边界、缓存语义、并发控制和性能归因形成的完整实现体系。

8 已知问题与后续工作

对于尚未完成或仍然存在风险的部分，本项目选择在报告中直接说明当前状态和后续计划。这样既可以避免将尚未闭合的能力误写为已经完成，也可以为后续开发和验证提供明确入口。

主要已知问题及后续计划如表 8-1 所示。

表 8-1 已知问题与后续工作

问题	当前状态	后续计划
JBD2 revoke 尚未写盘	恢复侧已经能够读取 revoke block，但写侧尚未生成并持久化 revoke 记录。在特定的元数据块释放、复用和崩溃时序下，理论上仍存在旧日志覆盖新数据的风险。	实现 revoke block 的生成、去重与写入，并在 recovery 中按照事务序列号过滤；增加元数据块释放后重新作为数据块使用的崩溃测试。
ENOSPC 半成品元数据	满盘和空间分配中途失败场景下，只读降级、分配回滚和 fsck 验证仍需进一步完善。	在不可安全继续写入时将文件系统切换为只读保护，完善 <code>OperationAllocGuard</code> 回滚，并增加 <code>e2fsck</code> 门控。
SQLite 性能差距	当前测试的性能仍不理想，性能差距的归因仍需要具体定位和分析。	优先定位并解决途中写回导致的数据损坏，随后实现安全后台 <code>writeback</code> 、脏页节流、空间预留和 <code>delalloc</code> 。
official xfstests 全量测试	核心子集已经做到 0 FAIL，但部分测试由于环境依赖、未实现功能或更底层问题尚未完全验证。	分类处理 NOTRUN、测试环境依赖和真实功能缺陷，补充 <code>hardlink</code> 、 <code>symlink</code> 、 <code>xattr</code> 及相关运行环境。
进一步的性能优化	<code>fio</code> 顺序读写和并发覆盖写已经取得较好结果，但真实应用仍有提升空间。	继续探索适合 Rust 和 Asterinas <code>framekernel</code> 架构的缓存、写回、空间分配和异步 I/O 优化方法。

上述问题中，`revoke` 和 `ENOSPC` 主要影响系统健壮性与边界完整性，而安全后台写回和 `delalloc` 则决定 `SQLite` 等真实应用负载是否能够进一步接近 Linux EXT4。因此，后续工作应继续坚持“正确性优先、性能优化建立在可验证语义之上”的原则。

9 开发过程与工程管理

9.1 开发路线

本项目并非一次性完成，而是按照功能正确性、崩溃一致性、缓存集成、性能归因和性能优化等阶段逐步推进。

项目早期的重点是使 EXT4 能够在 Asterinas 上正常挂载，并完成基础文件读写、目录操作、stat 和 truncate 等 POSIX 功能。随后逐步补充多块 Extent 管理、JBD2 日志、崩溃恢复、PageCache、mmap 和 O_DIRECT。进入后期后，主要工作转向性能归因、针对性优化和正确性测试。整体开发路线如图 9-1 所示。

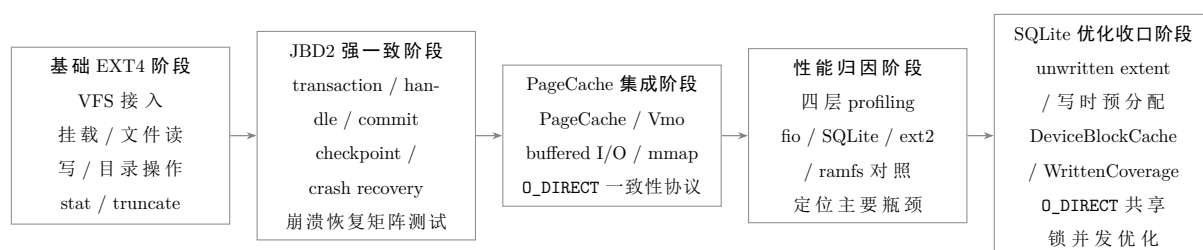


图 9-1 项目分阶段开发流程

这样的开发方式虽然需要较长的验证周期，但更适合文件系统项目。文件系统错误往往不会在功能首次运行时立即表现为崩溃，而可能在系统重启、fsync、并发访问、满盘压力或崩溃恢复阶段才暴露。

项目早期也曾出现“功能刚刚跑通便开始优化”的问题。部分优化从直觉上看较为合理，例如尽量推迟分配、减少写回次数或将更多状态长期保存在内存中，但在 SQLite 和 fsstress 等真实压力负载下，可能引发数据损坏、内存耗尽或长尾 panic。

经过多轮实践，项目逐渐形成了较为稳定的开发纪律：

1. 首先确保目标功能的语义和错误处理正确；
2. 使用 profiling 和对照实验定位具体瓶颈；
3. 每轮只处理一个明确且可测量的问题；
4. 优化完成后执行崩溃恢复、并发、fsync 和完整正确性测试；
5. 对负收益或存在正确性风险的方案及时回退并记录原因。

9.2 代码组织与可维护性

为了避免后期功能和优化改动相互污染，项目尽量按照职责将不同类型的逻辑放置在不同层次。

与 EXT4 和 JBD2 磁盘格式相关的定义集中在 `ext4_defs`；真正修改文件系统结构的操作集中在 `ext4_impls`；平台相关的 PageCache、bio、锁、VFS 接口和启动参数则集中在 `kernel/src/fs/ext4`。

这种组织方式有助于在调试时快速判断问题所在层次：

- 如果 `e2fsck` 报告盘面结构错误，优先检查 EXT4 核心库；
- 如果 `0_DIRECT` 与 buffered I/O 读取结果不一致，优先检查内核集成层的写回和失效协议；
- 如果崩溃后事务重放异常，优先检查 JBD2 提交与 recovery 路径。

缓存失效是系统可维护性中最容易变得混乱的部分。项目没有让每类缓存分别在任意路径中自行判断失效时机，而是尽量将失效操作挂接在少数统一收敛点上。

例如，受日志保护的元数据修改统一经过 `run_journaled_ext4`；`truncate`、`unlink` 和 `rename-overwrite` 等可能移除或改变映射的操作会清理 `WrittenCoverage`；`0_DIRECT` 数据 bio 绕过 `DeviceBlockCache` 时，必须在写入成功后显式执行 `invalidation`。

这种方式有时比较保守，可能牺牲部分缓存命中率，但能够降低旧映射、旧元数据或旧数据页继续参与正确性判断的风险。

项目文档与代码也尽量保持同步。仓库中的阶段性报告记录了每个阶段的设计动机、代码改动、测试结果和性能变化；本次初赛报告则将这些过程材料整理为更适合整体阅读和评审的版本。

9.3 遇到的典型问题

项目开发过程中遇到的主要问题，并不只是单个函数实现错误，而是文件系统语义、平台基础设施和性能目标之间的相互影响。

第一个典型问题是 SQLite 性能非常低，但项目初期无法确定主要时间消耗在哪里。直觉上可能认为是 Rust 语言较慢、Asterinas 平台较慢，或者 JBD2 日志机制本身过于昂贵。

通过 EXT4、EXT2 和 `ramfs` 三个文件系统对照发现，EXT2 和 `ramfs` 在相同平台上均接近 Linux 对照性能，说明平台并不是主要瓶颈。进一步的 profiling 表明，覆盖写快速路径仍然在反复读取 Extent 树块。这一结果改变了后续优化方向：项目没有首

先重写 JBD2，而是优先实现设备块缓存、WrittenCoverage 和批量写回。

第二个典型问题是 delalloc。从理论上讲，延迟分配是最接近 Linux EXT4 的优化方向，因为它能够将逐页产生的元数据开销摊薄到更大的 Extent 上。但在实际实现中出现了两个问题：一是取消逐写检查后，大量脏页在内存中积累并最终导致 OOM；二是加入脏页节流后，非 fsync 时刻的途中写回会导致 SQLite 数据损坏。

这一结果说明，delalloc 并不是修改几个常量即可完成的功能，而是需要后台 write-back、空间预留、写回失败处理和一致性协议共同配合的系统级机制。

第三个问题是并发写入性能无法扩展。fio 的 numjobs 增加后，同文件写入带宽仍接近单流水平。代码审查发现，per-inode 互斥锁覆盖了完整的 O_DIRECT 路径，包括设备等待时间，从而将原本可以并发的纯覆盖写完全串行化。

项目参考 Linux shared i_rwsem 的思路，将不会改变文件映射和文件大小的 O_DIRECT 纯覆盖写放到共享锁下执行，才获得明显的并发性能提升。

典型问题的定位与处理过程如表 9-1 所示。

表 9-1 开发过程中遇到的典型问题

问题	最初现象	定位方法	最终处理
SQLite 性能低	speedtest1 仅达到 Linux EXT4 的 2.97%。	三个文件系统对照与四层 profiling。	增加设备块缓存、覆盖写快速路径、预分配和批量写回。
delalloc 实现受阻	去除逐写检查后出现 OOM，加入节流后出现数据损坏。	Stage 0/1a 隔离实验与 SQLite 完整性验证。	回退当前实现，等待安全后台写回基础设施完善。
并发写无法扩展	numjobs 增加后仍维持在约 2800 MB/s。	fio F 组测试与锁持有范围代码审查。	使用 per-inode RwMutex 和共享锁覆盖写路径。
满盘压力下 panic	fsstress 和 ENOSPC 场景中低频触发异常。	日志调用栈、Extent 节点和空间分配状态检查。	增加边界钳制，将部分损坏场景降级为 EIO。

9.4 文档与代码版本管理

本次初赛报告中的性能数据、测试结果和已知边界，均以初赛最终提交代码版本为准。为了避免不同阶段数据混用，本文保留以下统一测试口径：

1. 所有正式性能结果默认关闭 `direct_read_cache`;
2. SQLite 数据采用 `page_cache=1` 的真实应用路径;
3. fio 基础测试采用 `direct=1`、`numjobs=1`;

通过明确实验配置、代码版本和数据来源，可以避免历史测试结果、临时 profiling 数据和最终性能结论之间发生混淆。

10 AI 使用说明

本章按赛事要求披露项目开发过程中 AI 工具的使用情况，包括工具与模型名称、使用场景、人机分工、AI 辅助形成的材料，以及可追溯的交互记录。

赛题：2026 全国大学生计算机系统能力大赛·操作系统设计赛·OS 功能挑战赛道。

项目：在 Asterinas (Rust framekernel) 上用 Rust 实现兼容 POSIX、支持 Extent 与 JBD2 日志机制的 EXT4 文件系统，并探索面向 RustOS 架构的性能优化技术。

文档日期：2026-06-22。

10.1 披露范围

本项目使用 AI 的基本方式是：参赛队负责需求拆分、功能实现、代码修改、测试取舍、结果验收和最终提交；AI 工具主要用于沟通实现思路、检查代码风险、排查运行 bug、生成自动化测试脚本、整理测试数据表格，以及总结阶段状态供队内讨论。

10.2 AI 工具与模型清单

项	内容
交互与执行工具	Claude Code (命令行交互与执行 harness)
使用的大模型	DeepSeek V4 Pro
使用周期	2026-03 至 2026-06
使用方式	本地终端交互

10.3 使用场景

AI 工具主要用于以下辅助场景：

- **思路沟通与代码检查**：围绕参赛队已经实现或准备实现的功能，讨论可能影响的模块、边界条件和风险点，用作人工检查清单。
- **运行问题排查**：当功能运行不正确、测试失败或出现 panic/timeout 时，辅助分析日志和报错信息，缩小 bug 排查范围。
- **自动化测试辅助**：由于 xfstests、crash recovery、SQLite、fio 等测试耗时较长，AI 辅助编写和维护测试脚本、整理运行命令，并按参赛队要求执行重复测试。

- **测试数据整理：**将多轮 benchmark、profile、PASS/FAIL 结果整理为表格，便于比较不同阶段的性能变化和正确性状态。
- **阶段状态总结：**根据已运行的测试结果和当前代码状态，总结阶段完成情况、遗留问题和下一步计划，方便队内沟通。

10.4 人机分工

10.4.1 参赛队负责的工作

- 确定阶段目标和优先级，例如 JBD2 日志、PageCache、O_DIRECT、SQLite 写性能等开发顺序。
- 完成功能实现和代码修改，包括 extent 映射、事务边界、dirty page 写回、fsync/fdatasync、并发锁和 crash recovery 相关逻辑。
- 审查代码改动是否符合项目设计，确认是否能够进入提交历史。
- 对性能结果做取舍，避免采用不能反映真实文件系统能力的缓存口径或临时数据。
- 决定最终提交内容，维护 Git 分支和 commit 记录。

10.4.2 AI 辅助的工作

- 根据报错日志、测试输出和运行现象辅助定位问题。
- 编写或调整测试脚本，帮助重复运行耗时测试。
- 把多轮测试数据整理成表格或阶段总结，便于队伍判断下一步方向。
- 对部分关键路径提供代码安全检查，供参赛队判断是否采纳。

10.5 交互记录与可追溯性

与 AI 的交互记录以文档的形式保存在 docs/AI_usage/。这些文档记录了各阶段的测试结果分析和阶段总结。

10.6 诚信声明

本文件如实披露了项目中使用的 AI 工具、模型名称和主要使用场景。AI 参与的是思路沟通、代码问题检查、bug 排查、自动化测试脚本、测试运行辅助、数据表格整理和文档总结等工作；参赛队负责功能实现、核心设计、代码审查、测试验收和最终提交。

11 总结

本项目在 Asterinas 上实现了一个具备主体功能、可测试且具有明确设计边界的 Rust EXT4 文件系统。

在功能方面，系统支持 POSIX 核心接口、Extent 管理、JBD2 日志、PageCache、mmap、O_DIRECT 和多文件并发访问。在正确性方面，崩溃恢复、fsync 持久化、并发读写和页缓存一致性均具有独立测试证据。

在性能方面，`fiio O_DIRECT` 顺序读写已经能够追平或部分超过 Linux EXT4；SQLite 真实应用虽然仍未达到预期目标，但相对于初始版本已经获得约 8.6 倍提升。通过共享锁覆盖写路径，同文件并发写中的主要锁瓶颈也得到缓解。

本项目最大的收获并不是某一个单独的性能数字，而是形成了一套较为完整的系统性能研究过程：首先建立正确性基线，随后进行测量和归因，再围绕明确瓶颈实施优化，最后通过崩溃、并发、fsync 和完整性测试进行守底。

在这一过程中，部分看似有希望的方向最终被实验结果否定；也有一些最初并不起眼的结构，例如 DeviceBlockCache 和 WrittenCoverage，最终成为提升性能的重要机制。文件系统开发既需要性能意识，也需要对持久化语义、缓存一致性和失败边界保持谨慎。

如果继续推进，最值得优先完成的工作包括 JBD2 revoke 写入、ENOSPC 只读降级以及安全后台 writeback 与 delalloc。前两项能够进一步提高系统健壮性，后一项则可能使 SQLite 等真实应用负载真正接近 Linux EXT4 的高比例性能区间。

当前版本已经具备初赛展示所需的主要功能、测试数据和研究叙事基础，同时对尚未完成的边界进行了明确说明。

附录 A 核心实现细节补充

A.1 `run_journaled_ext4` 收敛点

`run_journaled_ext4` 是项目中非常关键的统一收敛点。所有会修改 EXT4 元数据的操作，包括写入过程中产生的块分配、truncate、目录创建、目录删除、rename 和 inode 元数据更新时间等，都需要通过该入口进入核心库。

该入口的目的并不只是封装函数调用，而是将对象锁、JBD2 事务、缓存失效、inode-tid 追踪、性能统计和错误处理放在同一位置统一执行。

如果没有这一收敛点，缓存失效会非常难以维护。例如，某个路径修改了 Extent，但忘记清理 Extent map cache；另一个路径释放了磁盘块，却没有清理 WrittenCoverage；还有路径更新了 inode size，但没有记录 fsync 需要等待的事务号。

这类错误在普通读写中可能不会立即暴露，但在崩溃恢复、缓存混合访问或并发压力下，会演化为难以定位的数据一致性问题。因此，`run_journaled_ext4` 的设计目标，就是减少分散在不同代码路径中的状态更新。

在一次 `journaled` 操作中，集成层首先获取相应对象锁，随后进入 `EXT4_RS_RUNTIME_LOCK` 和 EXT4 核心库操作。核心库完成元数据修改后，JBD2 handle 记录相应元数据块镜像，最后由集成层处理提交、缓存失效、事务号更新和性能统计。

这一模型牺牲了部分路径自由度，但换来了更加清晰和可验证的事务推理边界。

A.2 DeviceBlockCache 的位置选择

DeviceBlockCache 被放置在块设备 adapter 层，而不是放在 `ext4_rs` 中某个特定磁盘结构模块内。其主要原因是 SQLite 慢路径中反复读取的热点并不局限于某一种元数据。

inode table 块、Extent 树块、块组描述符和位图等都可能成为频繁访问对象。如果只缓存 Extent 查询结果，缓存容易受到文件碎片和失效粒度的限制；而缓存设备 home block，则可以统一承接多种元数据读取。

DeviceBlockCache 的位置还与 JBD2 overlay 有关。active transaction 中的元数据修改尚未写回 home block，此时直接从设备读取到的是旧 home 内容，必须叠加 JBD2 overlay 后才能得到当前最新视图。

因此，DeviceBlockCache 位于 overlay 之下，只缓存 home block 内容；overlay 仍然由上层根据事务状态动态叠加。checkpoint 或 recovery 写回 home block 时，会通过

adapter 执行 write-through，并同步更新缓存。

只有 `O_DIRECT` 数据 bio 会绕过 adapter 的普通 `write_offset` 路径，因此这一类直接写入必须显式失效对应设备块缓存。

主要设计选择如表 A-1 所示。

表 A-1 DeviceBlockCache 的设计选择

设计选择	原因	风险控制
缓存 home block 而非查询结果	不同类型的文件系统元数据均可受益，缓存覆盖面更广。	采用固定容量与 LRU 淘汰，未命中时重新访问设备。
放置在 JBD2 overlay 之下	保持 active transaction 读取时继续叠加 overlay 的语义。	checkpoint 和 recovery 通过 adapter write-through 更新缓存。
<code>O_DIRECT</code> 后显式失效	直接数据 bio 绕过 adapter，可能修改缓存对应的物理块。	<code>submit_direct_write_mappings</code> 完成后调用范围失效接口。

A.3 WrittenCoverage 的不变量

WrittenCoverage 的核心不变量是：

$$\text{coverage} \subseteq \text{truth}.$$

也就是说，缓存中记录为“已经写入”的区间，必须真实存在于 EXT4 的 written 映射中，但允许缓存少记。

少记的后果只是后续请求无法命中快速路径，需要重新执行 Extent 查询；多记的后果则可能破坏正确性。如果将文件空洞或 unwritten 区间错误判断为 written，覆盖写快速路径就可能绕过 journaled prepare，将数据写入不存在或不应直接覆盖的位置。

为了维护这一不变量，系统只允许以下两类操作扩展 WrittenCoverage：

1. 通过权威的整文件映射遍历进行 populate；
2. journaled prepare 成功后，将本次已经建立并确认 written 的映射插入 coverage。

相反，凡是可能移除映射或改变映射含义的操作，例如 truncate、unlink、rename-overwrite 和 Extent 转换，都直接清理对应 inode 的 coverage。

该策略比较保守，但适合在比赛阶段控制正确性风险。WrittenCoverage 对 SQLite 的主要帮助体现在覆盖写和反复更新场景中：原先每次写入都可能调用映射查询并下沉 Extent 树，现在如果目标区间已经被 coverage 覆盖，只需要执行一次 BTreeMap 前驱查询。

A.4 fsync 链路的设计取舍

fsync 是文件系统中容易被错误优化的路径之一。从性能角度看，fsync 执行的工作越少，调用耗时越短；但从正确性角度看，fsync 必须保证应用已经确认持久化的数据在系统崩溃后仍然存在。

本项目对 fsync 的基本原则是：可以消除冗余工作，但不能改变应用可观察到的持久化语义。

早期实现中，fsync 在完成脏页写回后，还会 decommit 整个文件的 VMO 区间，相当于将已经写回的 clean 页面也全部丢弃。这一行为并不是 fsync 语义所要求的，Linux fsync 通常也不会主动清空所有 clean 页。

保留 clean 页面后，SQLite 的后续读取和部分页更新可以继续命中内存，避免每次事务提交后都从设备重新构建整个工作集。该优化本质上是删除不必要的工作，而不是放宽 crash consistency。

批量写回也采用相同思路。在 ordered 模式下，普通文件数据本身不进入 journal，JBD2 保护的是元数据。对于已经分配完成的连续区间，脏页可以按照连续 run 合并写回，减少逐页 handle、映射查询和 4 KiB bio 提交。

与此同时，commit block 之前的设备 sync 和 fsync 末尾的最终 flush 仍然保留，因此应用看到的持久化边界没有发生变化。

附录 B 复现命令与证据索引

B.1 常用复现命令

下面列出本地复查时常用的命令。实际运行需要 Linux 宿主、Docker、/dev/kvm 以及对应版本的 Asterinas 容器镜像。

崩溃恢复矩阵：

```
1 PHASE4_DOCKER_MODE=crash_only ENABLE_KVM=1 \
2 BENCH_ENABLE_KVM=1 BENCH_ASTER_NETDEV=tap \
3 BENCH_ASTER_VHOST=on \
```

```
4 bash tools/ext4/run_phase4_in_docker.sh
```

O_DIRECT 守底测试，关闭投机缓存和页缓存：

```
1 EXT4_DIRECT_READ_CACHE=0 EXT4_PAGE_CACHE=0 \
2 LOG_LEVEL=error BENCH_ENABLE_KVM=1 \
3 BENCH_ASTER_NETDEV=tap BENCH_ASTER_VHOST=on \
4 bash test/initramfs/src/benchmark/fio/run_ext4_summary.sh
```

SQLite 真实应用测试：

```
1 FS_LIST=ext4 PAGE_CACHE_LIST=1 LOG_LEVEL=error \
2 bash test/initramfs/src/benchmark/sqlite/run_sqlite_summary.sh
```

fio 参数 sweep，仅运行并发组：

```
1 SWEEP_GROUPS=F RUN_G_CORRECTNESS=0 \
2 bash test/initramfs/src/benchmark/fio/run_parameter_sweep_summary.sh
```

B.2 测试运行日志

本节不附带任何分析类文档，仅索引测试脚本实际运行时产生的原始日志。附录 B.1 中的各脚本在运行结束后，会输出对应测试的 PASS/FAIL 汇总、benchmark 数值与 profiling 记录，这些运行日志即为本项目正确性与性能结论的直接证据，其对应关系如表 B-1 所示。

表 B-1 测试运行日志索引

测试类别	运行脚本	日志内容
崩溃恢复矩阵	tools/ext4/run_phase4_in_docker.sh	各 crash point 的 18 项 PASS/FAIL 结果，以及挂载时 JBD2 日志重放与恢复过程日志。
O_DIRECT 守底测试	.../fio/run_ext4_summary.sh	关闭投机缓存与页缓存后，fio 顺序读写各块大小的吞吐与延迟日志。

续下页

表 B-1 测试运行日志索引（续）

测试类别	运行脚本	日志内容
SQLite 真实应用	.../sqlite/run_sqlite_summary.sh	speedtest1 各轮墙钟时间，以及每轮测试后的 integrity_check 输出。
fio 参数 sweep	.../fio/run_parameter_sweep_summary.sh	96 个参数组合及 C1 并发复测的逐项吞吐日志。

参考资料

- [1] Avantika Mathur, Mingming Cao, Suparna Bhattacharya, Andreas Dilger, Alex Tomas, and Laurent Vivier. *The New EXT4 Filesystem: Current Status and Future Plans*. Proceedings of the Linux Symposium, Vol. 2, pp. 21–34, 2007. <https://www.kernel.org/doc/ols/2007/ols2007v2-pages-21-34.pdf>.
- [2] Linux Kernel Community. *EXT4 General Information*. Linux Kernel Documentation. <https://docs.kernel.org/admin-guide/ext4.html>.
- [3] Linux Kernel Community. *EXT4 Block and Inode Allocation Policy*. Linux Kernel Documentation. <https://docs.kernel.org/filesystems/ext4/allocators.html>.
- [4] Linux Kernel Community. *Overview of the Linux Virtual File System*. Linux Kernel Documentation. <https://docs.kernel.org/filesystems/vfs.html>.
- [5] Linux Kernel Community. *Linux EXT4 Source Code*. <https://elixir.bootlin.com/linux/latest/source/fs/ext4>.
- [6] Lanyue Lu, Andrea C. Arpaci-Dusseau, Remzi H. Arpaci-Dusseau, and Shan Lu. *A Study of Linux File System Evolution*. In Proceedings of the 11th USENIX Conference on File and Storage Technologies (FAST '13), pp. 31–44, 2013. <https://research.cs.wisc.edu/adsl/Publications/fsstudy-fast13.pdf>.
- [7] Samantha Miller, Kaiyuan Zhang, Mengqi Chen, Ryan Jennings, Ang Chen, Danyang Zhuo, and Thomas Anderson. *High Velocity Kernel File Systems with Bento*. In Proceedings of the 19th USENIX Conference on File and Storage Technologies (FAST '21), pp. 65–79, 2021. <https://www.usenix.org/conference/fast21/presentation/miller>.
- [8] Hayley LeBlanc, Nathan Taylor, James Bornholt, and Vijay Chidambaram. *SquirrelFS: Using the Rust Compiler to Check File-System Crash Consistency*. In Proceedings of the 18th USENIX Symposium on Operating Systems Design and Implementation (OSDI '24), pp. 387–404, 2024. <https://www.usenix.org/conference/osdi24/presentation/leblanc>.

-
- [9] Kevin Boos, Namitha Liyanage, Ramla Ijaz, and Lin Zhong. *Theseus: An Experiment in Operating System Structure and State Management*. In Proceedings of the 14th USENIX Symposium on Operating Systems Design and Implementation (OSDI '20), pp. 1–19, 2020. <https://www.usenix.org/conference/osdi20/presentation/boos>.
 - [10] Yuke Peng, Hongliang Tian, Junyang Zhang, Ruihan Li, Chengjun Chen, Jianfeng Jiang, Jinyi Xian, Xiaolin Wang, Chenren Xu, Diyu Zhou, Yingwei Luo, Shoumeng Yan, and Yinqian Zhang. *ASTERINAS: A Linux ABI-Compatible, Rust-Based Framekernel OS with a Small and Sound TCB*. In Proceedings of the 2025 USENIX Annual Technical Conference (USENIX ATC '25), pp. 307–323, 2025. <https://www.usenix.org/conference/atc25/presentation/peng-yuke>.
 - [11] Asterinas Team. *Asterinas GitHub Repository*. <https://github.com/asterinas/asterinas>.
 - [12] yuoo655. *ext4_rs: An OS-Independent Rust EXT4 File System*. https://github.com/yuoo655/ext4_rs.